

AI ZERO TO HERO | SYSTEM DESIGN SERIES

# Breaking Big Architectures

*10 giants, taken apart from one server to planet scale.*

Analogies first. Diagrams everywhere. Engineer words explained.

Netflix · Uber · Telegram · Amazon · Google Search · Spotify · Airbnb · Discord · Stripe · Zoom

**Sheharyar Sarmad**

linkedin.com/in/sheharyar-sarmad-9b7736289

github.com/Sheharyar-Sarmad/ai-zero-to-hero

WELCOME

# Breaking Big Architectures

Every app you love started as **one server and one database**. This book takes ten famous products and walks them from that humble start to the planet-scale systems behind them, one painful problem and one clever fix at a time. For each company you will see the analogy first, then the plain-English story, then a **scale ladder**, a full architecture diagram and deep dives where the plain words are translated into the technical terms engineers use.

This is part of my **AI Zero to Hero** series and the companion to **Module 25: System Design Foundation**. Instagram, Twitter (X), WhatsApp and YouTube are already covered there, so this book focuses on **ten new giants**, including the three most requested ones: **Netflix, Uber and Telegram**.

| # | Teardown | The big lesson |
|---|---|---|
| **1** | Netflix | Separate the brain (cloud) from the muscle (edge boxes inside ISPs) |
| **2** | Uber | Treat the city as a grid of cells and keep live state in memory |
| **3** | Telegram | Store once, sync by bookmark, and pull for huge audiences |
| **4** | Amazon | Teams own services; carts favour availability, money favours consistency |
| **5** | Google Search | Split the index, ask everyone, merge, and tame the slowest machine |
| **6** | Spotify | Chunk, prefetch and precompute so everything feels instant |
| **7** | Airbnb | Retrieve then rank; never sell the same night twice |
| **8** | Discord | One lightweight process per community; bucket your data |
| **9** | Stripe | Make every write safe to repeat and every cent traceable |
| **10** | Zoom | Forward, do not mix; late is worse than lost |

| Resource | Link |
|---|---|
| **Companion module** | 25_systemdesign_foundation |
| **Full repository** | github.com/Sheharyar-Sarmad/ai-zero-to-hero |
| **Connect with me** | Sheharyar Sarmad on LinkedIn |

> **REMEMBER**
> **A note on accuracy.** These are simplified, educational reconstructions based on public engineering blogs, papers and talks. Companies change their internals all the time, and numbers are approximate and may be dated. Learn the **patterns**, not the exact blueprints.

HOW TO READ

# Anatomy of a teardown

Every company chapter has the same shape, so you always know where you are:

| Section | What it gives you |
|---|---|
| **Banner** | Four quick facts: what it is, the hardest problem, what it is famous for, and the core idea. |
| **Think of it like this** | An everyday analogy so the idea clicks before any jargon. |
| **Story in plain English** | What the product does and why it is hard, in simple words. |
| **Scale ladder** | Five steps from one server to planet scale, with what gets added at each step. |
| **What breaks, what gets added** | The pain at each stage, the fix, and the engineer's word for it. |
| **Big picture** | A layered architecture diagram of the high-end design. |
| **Deep dives** | Analogy, plain explanation, then the technical details underneath. |
| **Say it plain, say it technical** | A translation table from everyday words to engineer words. |
| **Decisions and their price** | Every choice has a cost. Here is what each one cost. |
| **What if things break?** | How the design survives failures and spikes. |
| **Check yourself and Redesign challenge** | Questions to answer out loud, plus a design problem to try. |

## The five mental models behind every architecture

| Mental model | In one line |
|---|---|
| **1. Everything is a trade-off** | Each win costs something; always name the price. |
| **2. Numbers decide** | Estimate traffic, storage and latency before choosing tools. |
| **3. Move state out** | Stateless servers scale; put memory in caches, stores and queues. |
| **4. Assume failure** | Machines, networks and regions fail. Design for it, rehearse it. |
| **5. Add one thing per pain** | Add a component only when a measured problem demands it. |

CONTENTS

# Table of contents

Breaking Big Architectures . . . 2
Anatomy of a teardown . . . 3
Table of contents . . . 4

**The Toolbox . . . 5**

Every system climbs the same ladder . . . 6

**The Ten Teardowns . . . 9**

01 Netflix . . . 10
02 Uber . . . 14
03 Telegram . . . 18
04 Amazon . . . 22
05 Google Search . . . 26
06 Spotify . . . 30
07 Airbnb . . . 34
08 Discord . . . 38
09 Stripe . . . 42
10 Zoom . . . 46

**Across All Ten . . . 50**

Which concept shows up where . . . 51
Side-by-side comparison . . . 52
Ten lessons from ten giants . . . 53
If you can answer these without hesitation, you are ready for the next round . . . 54
Borrow from the giants . . . 55

# START

## The Toolbox

*Prerequisite concepts you need before breaking big systems.*

- How every system grows: the scale ladder
- Traffic and scaling
- Speed: caches and CDNs
- Data: storage, replication, sharding
- Messaging and real-time
- Reliability patterns
- Self-test: are you ready?

PREREQUISITES

# Every system climbs the same ladder

Whatever the company, growth follows five steps. At each step something **breaks**, and engineers **add** one component to fix it. If you remember this ladder, every case study in this book becomes easy to follow.




**INTERVIEW TIP**

In any design conversation, say: 'Let me start with one server, then see what breaks as we grow.' Walking the ladder out loud shows structured thinking.

## How to use this toolbox

Each concept below has a plain-English picture, then the technical words you will hear in interviews and engineering blogs. The last column shows where it appears in the teardowns. If a term in a chapter ever feels fuzzy, come back to this section.

### 1. Traffic and scaling

| Concept | In plain English | Engineer words | Seen in |
|---|---|---|---|
| **Load balancer** | A host at the restaurant door who sends each group to a free table. | Layer 4 / Layer 7, round robin, least connections | All |
| **Horizontal scaling** | Open more shops instead of buying a bigger oven. | Scale out, stateless servers | All |
| **Stateless service** | Any teller can serve any customer because the file is in the shared cabinet. | No local state, tokens, external store | All |
| **Auto-scaling** | Call extra waiters when it gets busy, send them home when it is quiet. | Scale on metrics, cooldown, warm-up | ZOM, AMZ, NFX |
| **API gateway** | The reception desk that checks ID and routes visitors. | AuthN, routing, rate limits, TLS | AMZ, STR, ABB |
| **Rate limiting** | A bouncer letting in only so many per minute. | Token bucket, 429, Retry-After | STR, DSC, AMZ |

## 2. Speed: caches and CDNs

| Concept | In plain English | Engineer words | Seen in |
|---|---|---|---|
| **Cache** | A sticky note of answers you use daily instead of walking to the library. | Cache-aside, LRU, TTL, hit ratio | All |
| **CDN** | Corner shops near customers, stocked from a central kitchen. | Edge servers, origin, cache hit / miss | NFX, SPT, TGM, ABB |
| **Prefetching** | Bringing the bread basket before you ask. | Predictive loading, buffering | SPT, NFX |
| **Adaptive bitrate** | Choosing the picture quality your road can carry. | Segments, bitrate ladder, ABR | NFX, SPT, ZOM |

## 3. Data: storage, replication, sharding

| Concept | In plain English | Engineer words | Seen in |
|---|---|---|---|
| **SQL vs NoSQL** | A strict spreadsheet versus a toolbox of flexible stores. | Relational, key-value, document, wide-column | All |
| **Replication** | Photocopies of the master notes for readers and for safety. | Leader / follower, replication lag | AMZ, ABB, STR |
| **Sharding** | Splitting a giant library across buildings. | Shard key, hot shard, resharding | UBR, DSC, STR, ABB |
| **Consistent hashing** | Guests around a circular table owning the nearest dishes. | Hash ring, virtual nodes | UBR, DSC, AMZ |
| **Strong vs eventual consistency** | Everyone hears news instantly versus eventually. | ACID, CAP, quorum | STR, AMZ, NFX |
| **Inverted index** | The index at the back of a textbook. | Posting lists, tokenisation, BM25 | GGL, ABB, SPT |
| **Geospatial index** | Numbered map squares so you only check nearby ones. | Geohash, quadtree, H3, S2 | UBR, ABB |

## 4. Messaging and real-time

| Concept | In plain English | Engineer words | Seen in |
|---|---|---|---|
| **Message queue** | A ticket rail between waiters and cooks. | Producer, consumer, ack, DLQ | STR, AMZ, UBR |
| **Pub/sub and event log** | A radio broadcast everyone can tune into, with a replayable tape. | Topics, Kafka, consumer groups | UBR, SPT, NFX |
| **WebSocket** | A phone call that stays open instead of sending letters. | Persistent connection, heartbeat | DSC, TGM, UBR |
| **UDP real-time media** | A live radio show where late words are worse than lost ones. | UDP, jitter buffer, WebRTC | ZOM, DSC |
| **Fan-out** | One announcement delivered to many listeners. | Push (on write) vs pull (on read) | TGM, DSC, UBR |

## 5. Reliability patterns

| Concept | In plain English | Engineer words | Seen in |
|---|---|---|---|
| **Idempotency** | A lift button: pressing twice equals pressing once. | Idempotency key, dedupe | STR, AMZ, ABB |
| **Retries with backoff** | Redial a busy friend, waiting longer each time. | Exponential backoff, jitter | STR, AMZ |
| **Circuit breaker** | A fuse that stops calling a broken service. | Closed, open, half-open, fallback | NFX, AMZ, UBR |
| **Saga** | Book flight, hotel, car; cancel in reverse if one fails. | Compensating actions, outbox | AMZ, UBR, STR |
| **Multi-region** | Branches in several cities, sharing the load and the risk. | Active-active, failover, geo DNS | NFX, UBR, ZOM, GGL |
| **Chaos engineering** | Fire drills for software. | Failure injection, game days | NFX, AMZ |

## Self-test: are you ready for the teardowns?

**PREREQUISITE CHECK**

1. What is the difference between horizontal and vertical scaling?
2. Why must servers be stateless to scale out?
3. What is a cache hit ratio and why does it matter?
4. What problem does a CDN solve?
5. Why is a database index fast for reads but slow for writes?
6. What is replication lag and when does it hurt?
7. What is a shard key and what makes a good one?
8. Queue versus pub/sub: when do you use each?
9. What does idempotent mean? Give an example.
10. What does a circuit breaker do?

*Answer at least 8 of 10 without hesitation. If not, re-read the matching rows above or revisit Module 25.*

# 10
# The Ten Teardowns

*From one server to planet scale, one giant at a time.*

- Netflix
- Uber
- Telegram
- Amazon
- Google Search
- Spotify
- Airbnb
- Discord
- Stripe
- Zoom

ARCHITECTURE TEARDOWN

# Netflix

01

*Streaming video to hundreds of millions of screens without buffering.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Video streaming for hundreds of millions of members | Moving a huge amount of video smoothly, everywhere | Open Connect CDN and chaos engineering | Brain in the cloud, muscle at the edge |

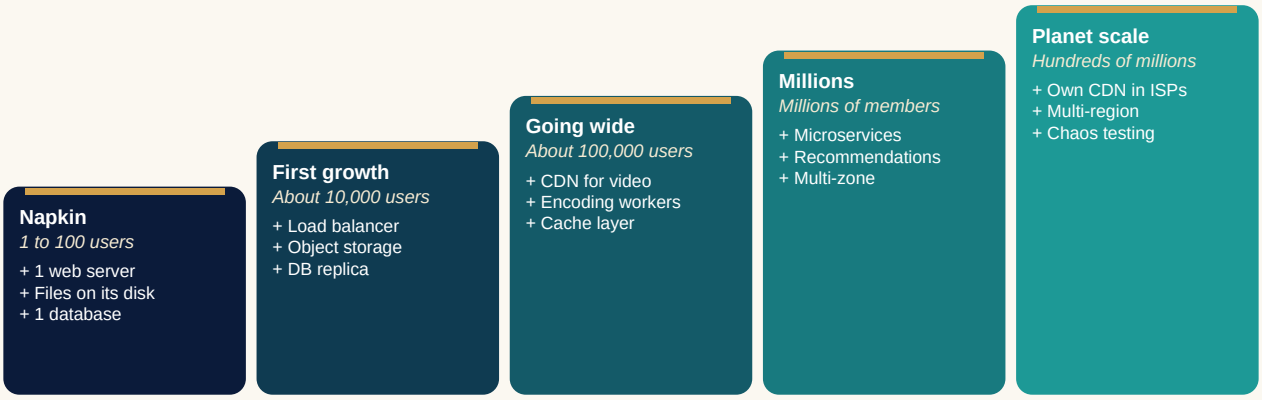
**THINK OF IT LIKE THIS**

A film library that does not make you travel. Instead of one giant cinema far away, Netflix places mini video shops inside your internet provider's building, stocks them overnight with what your neighbours will probably watch, and keeps only the directory ('which film, which shop') in the head office.

## The story in plain English

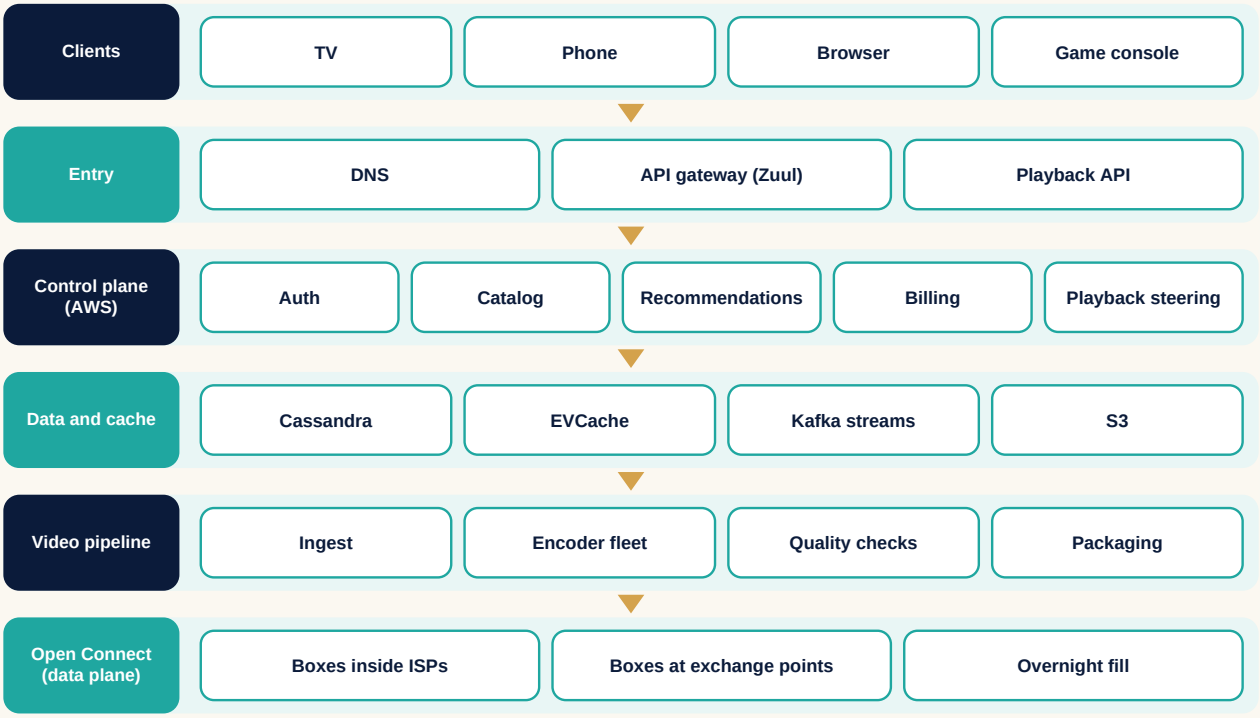
When you press play, two worlds work together. The **control plane** (the brain) runs on AWS: login, browsing, recommendations, billing and the decision of which file and which server to use. The **data plane** (the muscle) is Netflix's own CDN, **Open Connect**, which actually sends the video bytes. Keeping the brain and the muscle separate is the big idea: video is far too heavy to stream from a few central servers.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | One server cannot stream video to everyone | Video in object storage, load balancer, replicas | Object storage, horizontal scaling |
| **10K to 100K** | Video bytes crush bandwidth and cost | Serve video from servers near viewers | CDN / edge caching |
| **100K to millions** | One big codebase is slow to change; one failure hurts all | Split into small services, spread across zones | Microservices, multi-AZ |
| **Millions to planet** | Rented CDN is costly; whole regions can fail | Own CDN boxes inside ISPs, run in several regions, practise failures | Open Connect, active-active regions, chaos engineering |

## The big picture (high-end design)




*Simplified from public Netflix tech blogs. Names of internal tools change over time; the control plane / data plane split is the stable idea.*

## Deep dives

### Open Connect: video shops inside your ISP

> **ANALOGY**
> Corner shops stocked overnight with the films your neighbourhood will want, so nobody travels to the central warehouse.

**In plain English.** Netflix gives internet providers free storage servers. In quiet hours a 'fill' process copies the most popular titles onto them. When you press play, your device is pointed to the nearest box that has the title.

#### Under the hood

- Proactive caching (a push CDN): popularity is predictable, so content is placed before demand instead of waiting for cache misses.
- Steering: the playback service returns a ranked list of servers; the player can switch mid-stream if one becomes slow.
- Result: most traffic never leaves the viewer's own network, saving transit cost and cutting latency.

### Encoding: one film, hundreds of files

> **ANALOGY**
> Printing the same book in pocket, standard and large-print so every reader is comfortable.

**In plain English.** A source film is converted into many versions (resolutions, bitrates, codecs, audio languages) so any device on any connection gets a good fit. The work is cut into small chunks and encoded in parallel on a large fleet.

#### Under the hood

- Per-title and per-shot encoding tune the bitrate to scene complexity: a cartoon needs fewer bits than an action scene.

- Adaptive bitrate (ABR): the player downloads small segments and switches quality as bandwidth changes.
- Chunk-parallel encoding turns hours into minutes; failed chunks are retried independently (idempotent tasks).

### Resilience: break things on purpose

> **ANALOGY**
> Fire drills for software: practise the emergency so the real one is boring.

**In plain English.** Netflix assumes servers, zones and even regions will fail, so it tests this constantly. Services are written to degrade gracefully instead of crashing.

**Under the hood**

- Chaos Monkey randomly terminates instances during work hours so teams build for failure.
- Timeouts, circuit breakers and fallbacks: if personalisation fails, show a generic row of popular titles.
- Regional evacuation drills move traffic away from a whole AWS region and back.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Brain versus muscle* | **Control plane versus data plane** |
| *Mini shops inside your ISP* | **Open Connect appliances (OCAs)** |
| *Same film in many sizes* | **Adaptive bitrate ladder** |
| *Break things on purpose* | **Chaos engineering** |
| *Show something simpler when personalisation is down* | **Graceful degradation / fallback** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Build own CDN** | Video is the dominant cost and quality driver | Hardware, ISP partnerships and operations to run |
| **Cloud for the control plane** | Elastic, global, fast to build | Cloud bill and provider dependency |
| **Microservices** | Teams ship independently | Many network calls, harder debugging |
| **Cassandra for many datasets** | Availability and multi-region writes | Eventual consistency; model data around queries |

> **WHAT IF THINGS BREAK?**
> - A whole region fails: traffic is shifted to others; stateless services plus replicated data make this routine, and it is rehearsed.
> - A hit show premieres: titles are pre-filled to boxes before release so origin servers do not get a thundering herd.

**Toolbox concepts used here:** CDN, adaptive bitrate, microservices, circuit breaker, caching, Cassandra (eventual consistency), multi-region, chaos engineering

**CHECK YOURSELF**

1. Why separate the control plane from the data plane?
2. Why push content to boxes inside ISPs instead of using a normal CDN?
3. How does adaptive bitrate prevent buffering?
4. What does chaos engineering teach a team?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design the 'Continue Watching' row so it syncs across devices within seconds. Which store? Which consistency level? What if a user pauses on two devices at once?

ARCHITECTURE TEARDOWN

# Uber

02

*Matching riders and drivers in real time across the whole planet.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Ride hailing, delivery and freight marketplace | Real-time matching of moving drivers and riders | H3 hex grid, Ringpop, thousands of services | Treat the city as a grid of cells |

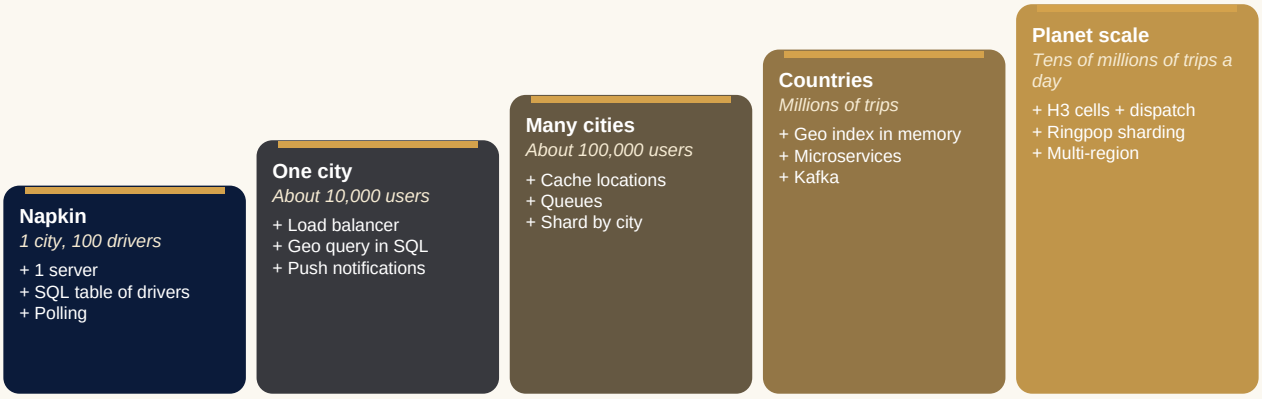
**THINK OF IT LIKE THIS**

A taxi dispatcher with a giant live map. Every driver is a moving dot, and the map is cut into small honeycomb cells. When someone asks for a ride, the dispatcher checks the nearby cells, compares arrival times and offers the trip to the best driver.

## The story in plain English

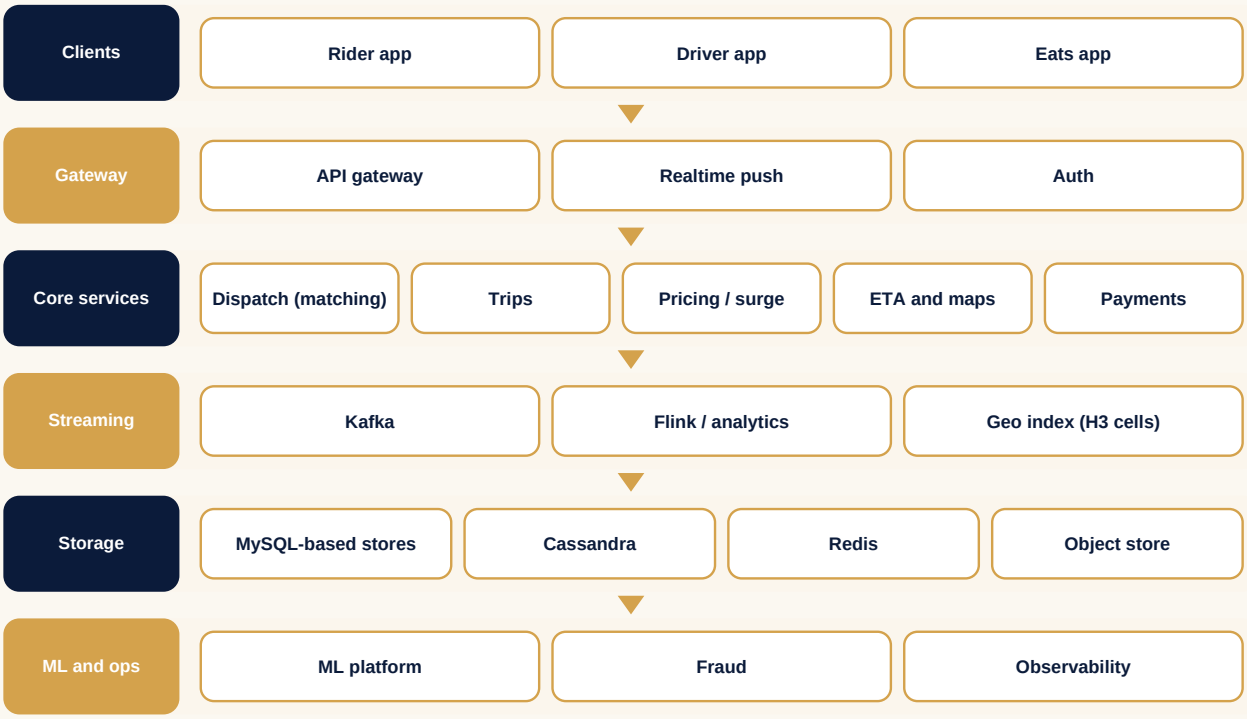
Uber is a two-sided real-time marketplace. Millions of phones send locations every few seconds, and when a request arrives the system must find, rank and reserve a nearby driver within a second or two without double-booking. Early Uber was a monolith; as cities and products multiplied it was split into many services.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Geo queries on a disk database are slow when drivers move constantly | Keep live locations in memory, not on disk | In-memory geospatial index |
| **10K to 100K** | One database and one codebase for every city | Shard by city; queues for trips and payments | Sharding, async messaging |
| **100K to millions** | The monolith slows teams; failures spread | Split into dispatch, pricing, payments, maps and ETA services | Microservices, service discovery |
| **Millions to planet** | Dispatch must scale with no central bottleneck | Cell-based sharding on a hash ring with gossip; stream processing for pricing | Ringpop, H3, Kafka and Flink |

## The big picture (high-end design)




*Simplified from public Uber engineering posts. Internal systems evolve; the cell-based, in-memory, service-oriented shape is the lasting lesson.*

## Deep dives

### Finding nearby drivers: the honeycomb map

> **ANALOGY**
> Divide the city into honeycomb cells. To find taxis near you, check your cell and the ring around it.

**In plain English.** Uber open-sourced H3, a hexagonal grid. Every location maps to a cell ID. Each driver's cell lives in an in-memory index, so 'who is near me?' becomes a few cell lookups instead of distance maths on millions of rows.

#### Under the hood

- Hexagons have equal-distance neighbours, so 'ring of nearby cells' queries behave uniformly (squares have awkward diagonals).
- Locations update every few seconds in memory and can be rebuilt from the stream, so losing a node is survivable.
- Candidates are ranked by ETA over the road network, not straight-line distance.

### Dispatch: matching without double-booking

> **ANALOGY**
> Offering a job to one worker at a time with a short timer, not shouting it to the whole room.

**In plain English.** The dispatch system looks at nearby drivers, can batch requests over a tiny window to find a better overall set of matches, and offers a trip to one driver at a time. Trip state lives in a durable state machine so a retry never creates two trips.

#### Under the hood

- Offers have timeouts; a decline moves to the next candidate.
- Batching improves average wait for everyone at the cost of a very small delay.

- State transitions (requested, matched, arrived, in trip, completed) are idempotent and stored durably.

### Sharding the real-time world: Ringpop

> **ANALOGY**
> Dispatchers sitting around a circular table, each owning a slice of the map. If one leaves, neighbours take over the slice.

**In plain English.** Services holding live state need to be partitioned and survive node loss. Uber built Ringpop: nodes sit on a consistent-hash ring, use a gossip protocol to know who is alive, and forward each request to the node that owns the key.

**Under the hood**

- Consistent hashing moves little data when scaling; gossip removes the need for a central coordinator.
- One owner per key (a city cell, a driver) keeps its in-memory state coherent.
- Trade-off: hand-off during membership changes needs careful state recovery.

## Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Honeycomb cells on the map* | **H3 hexagonal grid (geospatial index)** |
| *Offer the job to one driver with a timer* | **Dispatch with offer timeout** |
| *Circular table of dispatchers* | **Consistent hash ring (Ringpop)** |
| *Neighbours telling neighbours who is alive* | **Gossip / membership protocol** |
| *Price thermostat for busy times* | **Surge pricing from supply and demand streams** |

## Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Live locations in memory** | Updates are constant and old positions are worthless | Needs a rebuild strategy and costs RAM |
| **Split the monolith** | Teams and cities scale independently | Thousands of services to operate and observe |
| **Shard by geography** | Most traffic is local to a city | Cross-city trips and huge cities need care |
| **Rank by ETA, not distance** | Rivers and one-way streets fool straight lines | Needs a routing engine and live traffic |

> **WHAT IF THINGS BREAK?**
> - A data centre fails: apps reconnect to another region and the trip state is recovered; apps keep local state to resume.
> - A concert ends and 50,000 people request rides: surge pricing pulls in supply and eases demand while queues smooth the burst.

**Toolbox concepts used here:** geospatial index, consistent hashing, gossip, state machine, idempotency, Kafka, microservices, multi-region

**CHECK YOURSELF**

1. Why hexagons rather than squares?
2. How do you avoid two riders getting the same driver?
3. Why keep driver locations in memory instead of a SQL table?
4. What does gossip replace in Ringpop?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design 'share my live trip' for 10 million simultaneous trips. How do you push driver positions to followers cheaply?

ARCHITECTURE TEARDOWN

# Telegram

03

*A cloud notebook for chats: every device sees everything, even in channels of millions.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Cloud messaging with huge groups and channels | Syncing history across devices at huge scale | MTProto, cloud chats, giant channels | Server stores history, clients sync by bookmark |

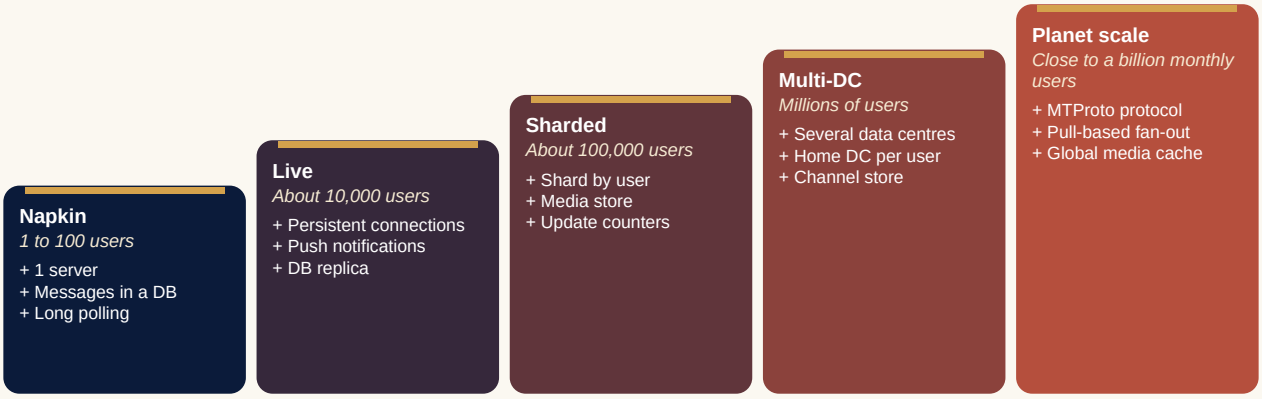
**THINK OF IT LIKE THIS**

A shared notebook in the cloud instead of letters on your phone. Every device you own reads the same notebook, with a bookmark that shows which pages it has already seen. Apps that deliver messages straight to each device behave more like letters.

## The story in plain English

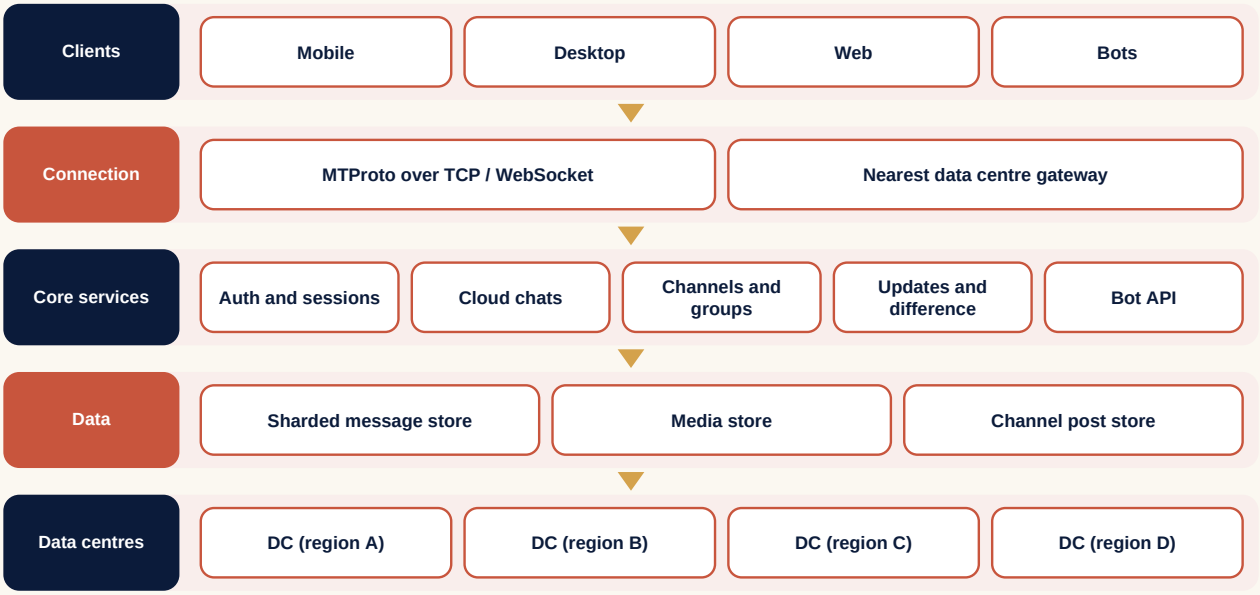
Telegram keeps your chat history on its servers (encrypted and spread across several data centres), so you can sign in on a new device and see everything. Because the server is the source of truth, devices sync by asking 'what happened since my bookmark?'. Giant channels work because a post is stored **once** and clients fetch it, instead of the server copying it into millions of inboxes.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Clients polling constantly wastes work | Keep a live connection and push updates | Persistent connection, push |
| **10K to 100K** | One database cannot hold every history | Shard messages by user or chat; media in separate storage | Sharding, object storage |
| **100K to millions** | Offline devices miss updates | Give state a counter; clients ask for the difference | Sequence counters (pts), delta sync |
| **Millions to planet** | Huge channels and worldwide latency | Store channel posts once, clients pull; several data centres | Fan-out on read, geo-distributed DCs |

## The big picture (high-end design)




*Simplified from Telegram's public protocol documentation and talks. Server code is closed, so internals beyond the documented protocol are inferred.*

## Deep dives

### Cloud chats versus secret chats

> **ANALOGY**
> A notebook in the cloud versus a sealed note that only two people hold the key to.

**In plain English.** Normal chats are encrypted between your device and Telegram's servers and stored there, which lets every device sync. Secret chats are end-to-end encrypted between two specific devices and are not stored in the cloud.

#### Under the hood

- Cloud chats: client-to-server encryption; enables multi-device sync, search and big groups.
- Secret chats: end-to-end encryption, one-to-one only, tied to a device, no cloud history.
- Trade-off: convenience and features versus the strongest privacy model.

### Sync with counters: pts, qts and seq

> **ANALOGY**
> Page numbers in the notebook. Your phone says 'I have read up to page 4210' and the server hands back page 4211 onward.

**In plain English.** Every account has update counters. A device that was offline asks for the difference since its last counters, and the server returns everything newer. A gap in the numbers reveals a missed update.

#### Under the hood

- Cheap, resumable and idempotent sync even after hours offline.
- Detects missed updates (gaps) and repairs them.
- The same idea as an event log with offsets.

## Giant channels: store once, pull many

> **ANALOGY**
> A notice board in the town square instead of posting a letter to every house.

**In plain English.** A channel post is saved once. Subscribers' apps fetch new posts when they open the channel, or get a light 'something new' ping. The server never copies a post into millions of inboxes.

**Under the hood**

- Fan-out on read avoids the celebrity write explosion.
- Media is stored once and cached near readers.
- View counters are approximate and aggregated asynchronously.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Notebook in the cloud* | **Cloud chats / server-side storage** |
| *Bookmark* | **Update counter (pts)** |
| *Ask what is new since my bookmark* | **Delta sync (getDifference)** |
| *Notice board in the square* | **Fan-out on read (pull)** |
| *Sealed note between two people* | **End-to-end encryption (secret chats)** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Server stores history** | Multi-device sync and search | The server holds sensitive data, so you must trust it more |
| **Custom protocol (MTProto)** | Tuned for speed on weak networks | Less battle-tested than TLS; debated by security experts |
| **Pull for big channels** | Avoids massive write storms | Slightly slower first load; needs caching |
| **Several data centres** | Latency and resilience; spreading jurisdictions | Some requests cross data centres |

> **WHAT IF THINGS BREAK?**
> - A data centre is unreachable: clients reconnect to another one; accounts have a home data centre so ownership is clear and data is replicated by design.
> - A viral channel post: reads hit caches and the post was written once, so there is no write storm.

**Toolbox concepts used here:** delta sync, event offsets, pull-based fan-out, sharding, geo-distributed data centres, object storage and cache, end-to-end encryption

**CHECK YOURSELF**

1. Why can Telegram sync a new device instantly while end-to-end-only apps struggle?
2. How does a counter-based sync detect missed updates?
3. Why is pull better for a 5-million-subscriber channel?
4. What does a secret chat give up?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design 'search my messages' across billions of cloud messages. Where does the index live and how do you keep it fresh?

ARCHITECTURE TEARDOWN

# Amazon

04

*A mall where every shop runs itself, and the cart must never lose an item.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Global e-commerce marketplace (and AWS) | Never losing an order during huge spikes | Service-oriented design, Dynamo, two-pizza teams | Every team owns a service with an API |

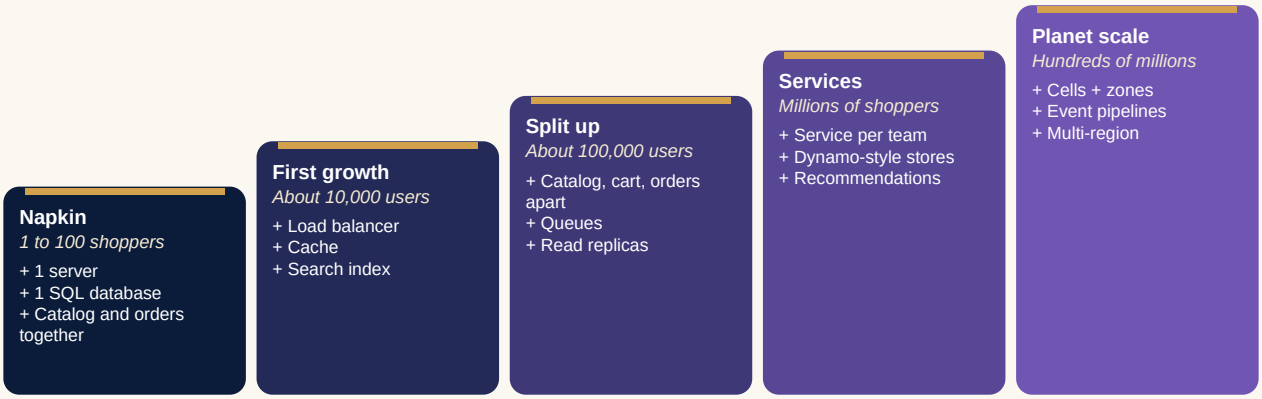
**THINK OF IT LIKE THIS**

A giant shopping mall where every shop (team) runs independently and talks to the others only through service counters (APIs), each with its own stockroom. If the shoe shop floods, the bookstore keeps selling.

## The story in plain English

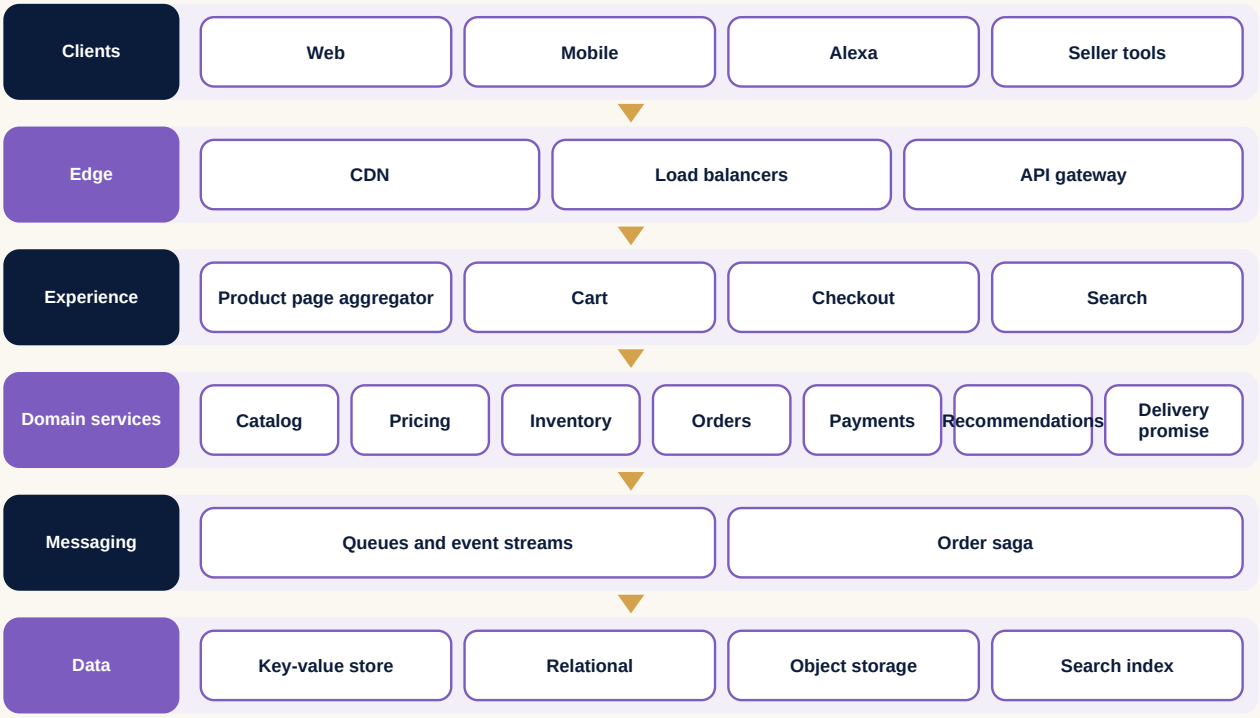
In the early 2000s Amazon's single big application was slowing hundreds of engineers. A famous internal rule required every team to expose its data and features through service interfaces. Today one product page triggers calls to dozens or even hundreds of services (price, stock, reviews, recommendations, delivery date), while the most critical flows, the cart and checkout, are built to stay available and never lose data.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Search and browsing slow the order database | Cache, search index, read replicas | Caching, search index, replication |
| **10K to 100K** | Order spikes and slow payments block browsing | Queue order processing; separate catalog, cart, orders | Async queues, service split |
| **100K to millions** | One codebase and schema block hundreds of teams | An API per team; key-value store for the cart | SOA / microservices, Dynamo-style store |
| **Millions to planet** | Failures must not spread; Prime Day spikes | Cells, multi-AZ headroom, load shedding, saga checkout | Cell architecture, static stability, saga + idempotency |

## The big picture (high-end design)




*Simplified from the public Dynamo paper, AWS Builders' Library articles and talks; individual services are far more numerous in reality.*

## Deep dives

### The shopping cart: always available

> **ANALOGY**
> A basket that always accepts an item, even if the till is down; duplicates get sorted out later.

**In plain English.** The Dynamo paper (2007) described a store built for the cart. It prefers being always writable over perfect consistency. If two copies of a cart disagree, they are merged and items added on either side are kept.

#### Under the hood

- Leaderless replication, quorums and consistent hashing.
- Conflicts resolved with vector clocks and merge-on-read.
- Philosophy: a rare extra item in the cart beats a refused add-to-cart.

### One product page, a hundred services

> **ANALOGY**
> A waiter collecting dishes from many stations, plating what is ready and skipping a station that is slow.

**In plain English.** The page is assembled by an aggregator that calls many services in parallel with strict time limits. If a non-essential widget (such as reviews) is slow, the page renders without it.

#### Under the hood

- Parallel calls, timeouts and fallbacks; hedged requests reduce tail latency.
- Caching at every layer: CDN, edge and service caches.
- Latency is tied directly to sales, so p99 is watched closely.

## Checkout as a saga with idempotency

> **ANALOGY**
> Booking a holiday of flight, hotel and car. If the car fails, cancel the others and refund.

**In plain English.** An order touches inventory reservation, payment authorisation, fraud checks and fulfilment, each owned by a different service. The order service runs these as steps with compensations, and every step is idempotent so retries never double-charge.

### Under the hood

- Order state machine persisted durably; steps coordinated by events and queues.
- Idempotency keys on payment and reservation calls.
- Inventory reservations expire if payment never completes.

## Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Every shop runs itself* | **Service-oriented architecture / microservices** |
| *Basket always accepts items* | **Always-writable store (AP, eventual consistency)** |
| *Merge two baskets* | **Conflict resolution / vector clocks** |
| *Skip a slow widget* | **Graceful degradation with timeouts** |
| *Cancel the hotel if the car fails* | **Saga with compensating actions** |

## Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Team boundaries are APIs** | Teams move independently | Dependency webs and many network hops |
| **Availability for the cart** | A refused add-to-cart is lost revenue | Occasional duplicate or resurrected items |
| **Strong consistency for payment and stock commit** | Money and inventory must be right | Lower throughput on hot items |
| **Cell-based architecture** | Limits the blast radius of failures | Duplicated capacity and routing complexity |

> **WHAT IF THINGS BREAK?**
> - An availability zone dies: tiers are over-provisioned across zones (static stability) so survival needs no scaling action.
> - Prime Day: capacity is pre-scaled, non-essential features are shed, and queues absorb bursts of orders.

**Toolbox concepts used here:** microservices, Dynamo-style store, quorum, saga, idempotency, caching, load shedding, multi-AZ

**CHECK YOURSELF**

1. Why was the cart built to favour availability over consistency?
2. How does a product page stay fast when 50 services are involved?
3. How does a saga undo a failed order?
4. What is static stability?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design the 'estimated delivery date' shown on the product page in under 50 ms. Which data, which cache and which fallback?

ARCHITECTURE TEARDOWN

# Google Search

05

*Searching the whole web in about a quarter of a second.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| **Web search over hundreds of billions of pages** | **Answering from the entire index in milliseconds** | **Inverted index, sharding, tail-latency tricks** | **Split the index, ask everyone, merge fast** |

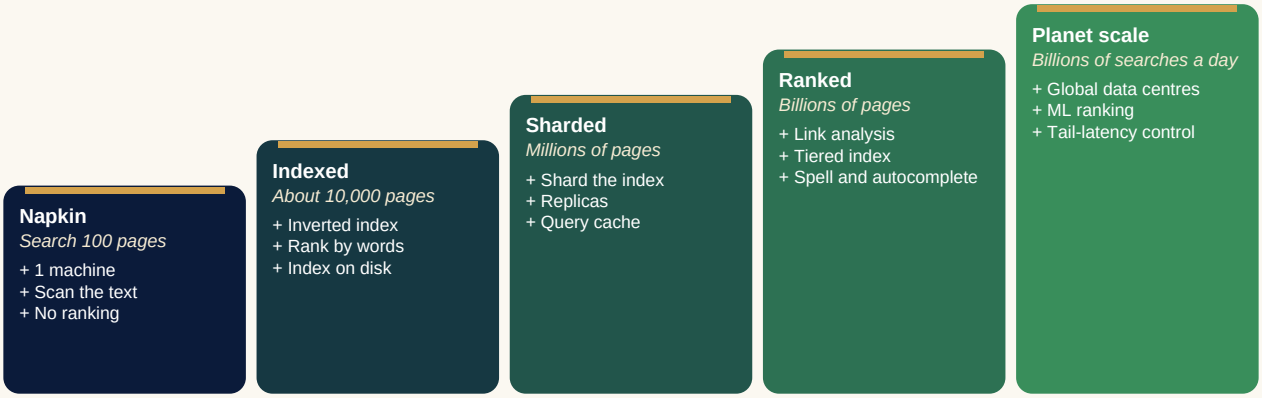
**THINK OF IT LIKE THIS**

A library with billions of books where the catalogue is cut into thousands of drawers. When you ask a question, a head librarian shouts it to every drawer-keeper at once; each returns their best five, and the head picks the overall best ten.

## The story in plain English

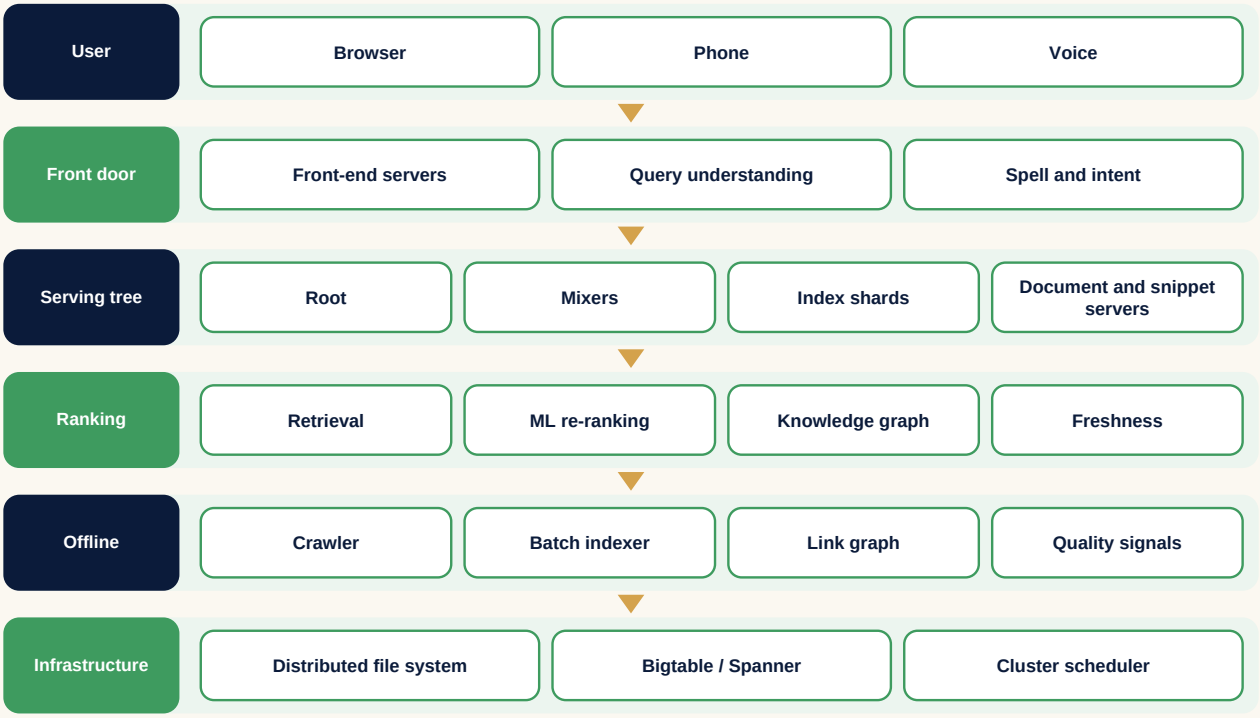
Google does three jobs. **Crawl**: robots fetch pages. **Index**: pages are turned into a giant word-to-page lookup. **Serve**: your query is sent to thousands of machines at once, each searching its own slice, and the results are merged, ranked and returned in a fraction of a second.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Scanning text per query is too slow | Build an inverted index (word to documents) | Inverted index |
| **10K to 100K** | The index no longer fits one machine | Split it across machines and replicate shards | Sharding, replication, scatter-gather |
| **100K to millions** | Many pages match; which is best? | Rank with links and many signals; cache popular queries; tier the index | PageRank, ranking models, caching, index tiers |
| **Millions to planet** | One slow shard slows the whole query | Duplicate slow requests, serve from nearby data centres, ML re-ranking | Hedged requests, multi-DC serving |

## The big picture (high-end design)




*Simplified from Google's published papers (MapReduce, Bigtable, Spanner, The Tail at Scale) and talks; the live system is far more elaborate.*

## Deep dives

### The inverted index

> **ANALOGY**
> The index at the back of a textbook, but for every word on the web.

**In plain English.** For each word, the index stores a list of documents that contain it (a posting list) and where. A two-word query intersects two lists.

#### Under the hood

- Compressed posting lists sorted by document ID make intersections fast.
- Word positions enable phrase queries; the best pages sit in a faster tier.
- A small, fresh index serves new content while the huge main index is rebuilt in the background.

### Scatter-gather and the tail at scale

> **ANALOGY**
> Shouting a question to a thousand people and answering after the 990th reply, ignoring stragglers.

**In plain English.** A query fans out to many shards, and its time is the time of the slowest one. Rare slow machines (garbage collection, noisy neighbours) would hurt every query, so tricks are used.

#### Under the hood

- Hedged requests: if the first replica is slow, send a duplicate to another and take whichever answers first.
- Deadlines and 'good enough' partial results instead of waiting for the slowest shard.
- Aggressive caching of popular queries.

## Ranking: from relevant to best

> **ANALOGY**
> Judges scoring a talent show on many criteria, then a final panel re-ranking the top contestants.

**In plain English.** Retrieval cheaply finds thousands of candidates. Heavier machine-learned models then re-rank the top few hundred using signals such as links (PageRank), freshness, language, location and quality.

**Under the hood**

- Two stages: cheap recall first, expensive precision second.
- Link analysis treats links as votes of trust.
- Quality is measured with human raters and live A/B experiments.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Index at the back of the book* | **Inverted index** |
| *Ask every drawer at once* | **Scatter-gather query** |
| *Send a second copy if the first is slow* | **Hedged request** |
| *Cheap shortlist, then careful judging* | **Two-stage retrieval and ranking** |
| *Links as votes* | **PageRank / link analysis** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Shard by document** | Each shard searches only its slice | Every query touches every shard |
| **Replicate shards** | Capacity and fault tolerance | Storage multiplied by the replica count |
| **Two-stage ranking** | Cheap recall plus costly precision | A good page missed in stage one cannot be rescued |
| **Accept slightly stale results** | Instantly indexing the whole web is impossible | Brand-new pages may take time to appear |

> **WHAT IF THINGS BREAK?**
> - A shard machine is slow or dies: replicas answer and hedging hides the delay.
> - A breaking-news spike: a fresh index and caches absorb it; trending queries are served from cache.

**Toolbox concepts used here:** inverted index, sharding, replication, scatter-gather, caching, hedged requests, two-stage ranking

**CHECK YOURSELF**

1. Why is a query's latency decided by the slowest shard?
2. What is a hedged request and what does it cost?
3. Why index in tiers?
4. Why rank in two stages?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design 'did you mean' spell correction for a billion queries a day. Where do suggestions come from and how fast must they be?

ARCHITECTURE TEARDOWN

# Spotify

06

*Instant music for hundreds of millions, plus a weekly mixtape that knows you.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| **Music and podcast streaming** | **Instant playback and personal discovery** | **Squads, Backstage, Discover Weekly** | **Stream small chunks fast; learn taste from behaviour** |

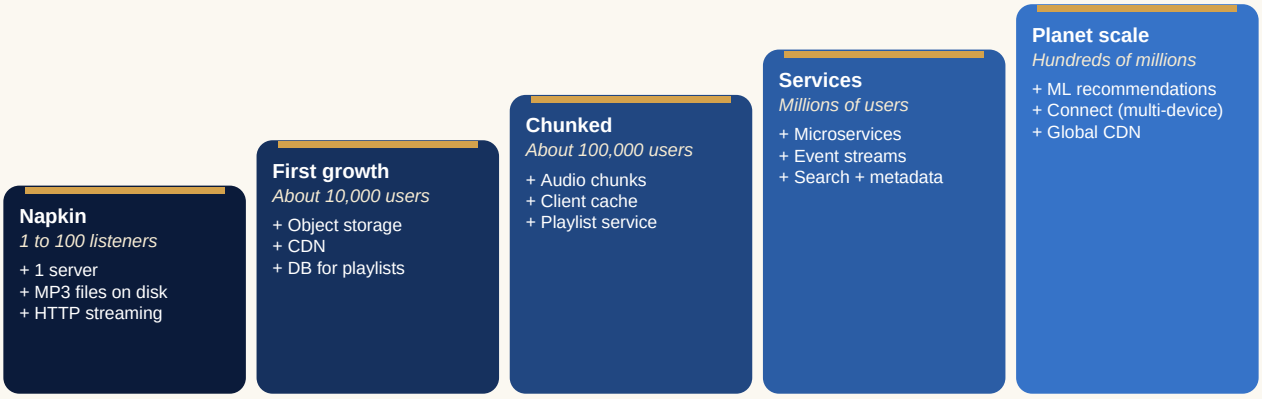
**THINK OF IT LIKE THIS**

A jukebox that already knows your taste. It keeps the first seconds of songs you are likely to play in a quick-grab tray, fetches the rest while you listen, and each week builds you a mixtape from people with similar tastes.

## The story in plain English

Spotify has two big problems. **Play**: audio must start almost instantly and never stutter. **Discover**: with over a hundred million tracks, suggest songs you will love. Playback relies on CDN-delivered audio chunks, prefetching and client caches; discovery relies on large data pipelines and machine learning. Spotify moved its backend to Google Cloud in the late 2010s.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Serving whole files from one server is slow and costly | Audio in object storage, files from a CDN | Object storage, CDN |
| **10K to 100K** | Downloads start slowly; skipping feels laggy | Stream in small chunks, prefetch next track, cache on device | Chunked streaming, prefetch, client cache |
| **100K to millions** | One codebase for playlists, search, social, payments | Independent services around events | Microservices, event streaming |
| **Millions to planet** | Which 30 songs out of 100 million? | Batch and stream pipelines feeding recommendation models | Collaborative filtering, embeddings, feature store |

## The big picture (high-end design)




*Simplified from Spotify Engineering blog posts and talks. Details such as vendors and internal tools have changed over the years.*

## Deep dives

### Instant playback: chunks, prefetch, cache

> **ANALOGY**
> A waiter who brings the bread basket first, so you never sit empty-handed.

**In plain English.** Songs are split into small chunks served from a CDN. The app prefetches the start of the next track and keeps a local cache, so pressing next feels instant. Quality adapts to the network.

#### Under the hood

- Chunked HTTP streaming: start playing after the first chunk arrives.
- Prefetching predicts the next track and downloads its first seconds early.
- Client-side cache means replays and offline mode cost no network.

### Discover Weekly: matching taste like friends do

> **ANALOGY**
> Finding your musical twin: people who like what you like probably like songs you have not heard yet.

**In plain English.** Collaborative filtering compares listening behaviour across millions of users. Language models read playlists and articles, and audio models analyse the sound itself. Everything is computed offline in batches and the per-user results are stored for quick serving.

#### Under the hood

- User and track embeddings (vectors) place similar items close together.
- Cold start: new tracks and users are handled with audio features and popularity until behaviour arrives.
- Compute offline, serve precomputed lists from a fast store.

## Spotify Connect: one remote, many speakers

> **ANALOGY**
> A TV remote that controls whichever TV you pick.

**In plain English.** Devices register with the backend. Commands such as play, pause or transfer go through the cloud to the chosen device, which fetches its own audio.

**Under the hood**

- Playback state is kept in the cloud so any device can take over.
- Persistent connections give low-latency control.
- The controlling phone is just a remote; audio does not pass through it.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Bread basket first* | **Prefetching** |
| *Bite-sized pieces* | **Chunked streaming** |
| *Musical twin* | **Collaborative filtering** |
| *Songs as points on a map* | **Embeddings (vector similarity)** |
| *Cloud remote control* | **Cloud-held playback state (Connect)** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Chunked streaming with prefetch** | Fast start and fewer stalls | Some wasted bandwidth when predictions are wrong |
| **Offline batch recommendations** | Cheap and consistent at huge scale | Recommendations lag behind your latest listening |
| **Microservices owned by squads** | Autonomy and speed | Platform tooling needed to tame the sprawl |
| **Cloud provider** | Elasticity and managed data tools | Provider dependency and cost management |

> **WHAT IF THINGS BREAK?**
> - A CDN node is slow: the client switches to another source and keeps its buffer, so the listener rarely notices.
> - A huge album drop: popular chunks are cached at the edge; hot metadata is cached to protect databases.

**Toolbox concepts used here:** CDN, caching, prefetching, microservices, event streaming, recommendations (ML), eventual consistency, multi-device state

**CHECK YOURSELF**

1. Why stream in chunks instead of whole files?
2. How does prefetch make 'next' feel instant?
3. Why compute Discover Weekly offline?
4. What is the cold-start problem?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design offline mode: keep 10,000 downloaded songs in sync across a phone and a laptop with limited storage.

ARCHITECTURE TEARDOWN

# Airbnb

07

*A hotel where every room is different, and no night is ever sold twice.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Marketplace for stays and experiences | Search and availability for one-of-a-kind listings | Rails monolith to services, Airflow | Search is retrieve then rank; booking never double-books |

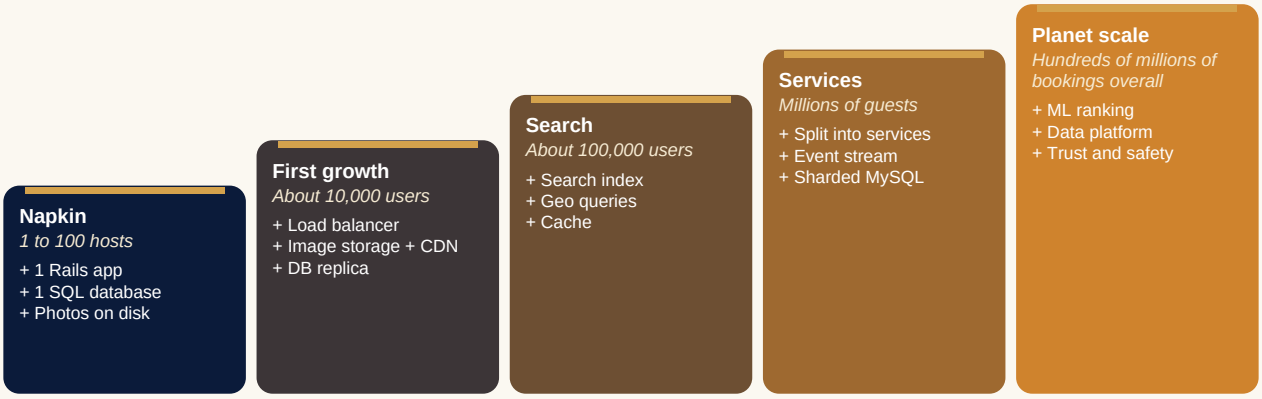
**THINK OF IT LIKE THIS**

A hotel where every room is unique and owned by a different person. The front desk must show only rooms free on your dates, rank them by what you will like, and make sure two guests never get the same room for the same night.

## The story in plain English

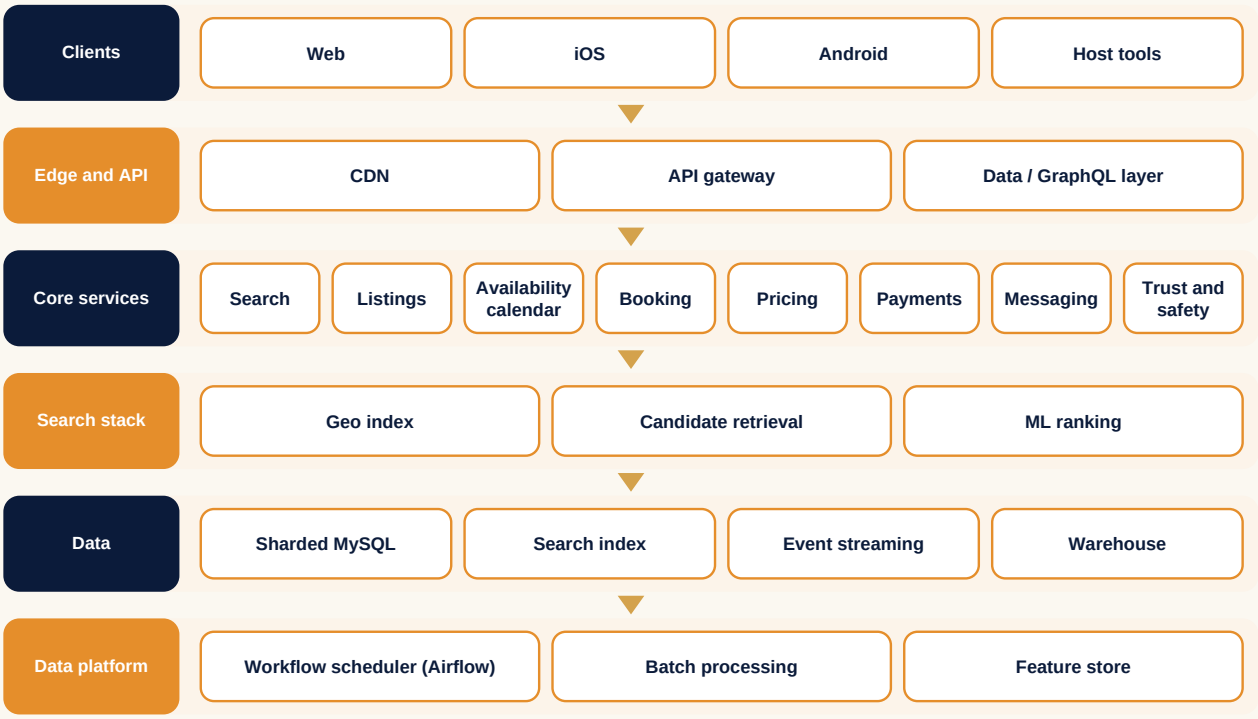
Unlike hotels with identical rooms, every listing has its own calendar, rules and price. Airbnb's search takes your dates, place and filters, finds candidate listings, ranks them with machine learning, and the booking system guarantees one guest per night using careful transactions. The company began with a Ruby on Rails monolith and moved to a service-oriented architecture.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Photos and pages slow down the app | Image storage with a CDN, replicas | Object storage, CDN, replication |
| **10K to 100K** | Searching by place, dates and filters in SQL is slow | Dedicated search index with geo support | Search index, geospatial queries |
| **100K to millions** | One codebase blocks teams; data outgrows one database | Service split; shard the data; stream events | SOA, sharding, event streaming |
| **Millions to planet** | Which of 10,000 homes should be first? | Two-stage retrieve and rank with machine learning; data platform for features | Retrieval + ranking, feature pipelines, A/B testing |

## The big picture (high-end design)




*Simplified from public Airbnb engineering posts. The company open-sourced Airflow; most other internals are described only in part.*

## Deep dives

### Search: retrieve, then rank

> **ANALOGY**
> A shop assistant who first fetches fifty promising items from the stockroom, then lays out the best ten on the counter.

**In plain English.** Retrieval narrows thousands of listings to a few hundred using location, dates, guests and filters. A machine-learned ranker then orders them using price, photos, reviews, host quality and your past behaviour.

#### Under the hood

- Geospatial retrieval on a map viewport; dates filter availability.
- Ranking models learn from bookings and are validated with A/B tests.
- Diversity and fairness rules prevent the top ten from looking identical.

### Never double-book: the calendar

> **ANALOGY**
> Writing on a shared wall calendar in pen, one person at a time.

**In plain English.** Availability is stored per listing per night. A booking runs in a transaction that checks the nights are free and marks them taken. Competing requests are serialised, and the loser is told the dates are gone.

#### Under the hood

- A uniqueness rule on (listing, night) or a row lock prevents two winners.
- A short hold during payment keeps dates reserved, expiring if unpaid.
- Syncing with external calendars is eventually consistent and is the classic source of conflicts.

## Payments across the world

**ANALOGY**
A currency desk and an escrow lawyer: take the money, hold it safely, release it at the right time.

**In plain English.** The guest pays at booking; funds are held and released to the host after check-in. Multiple currencies, taxes, refunds and fraud checks make this a state machine.

### Under the hood

- Idempotent payment requests prevent double charges.
- Ledger-style records track every movement of money.
- Risk scoring runs before and after booking.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Fetch fifty, show ten* | **Two-stage retrieval and ranking** |
| *Pen on a shared calendar* | **Transaction with uniqueness constraint** |
| *Holding dates while paying* | **Reservation hold with TTL** |
| *Money held until check-in* | **Escrow-style payment state machine** |
| *Test two versions on real users* | **A/B experimentation** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Rails monolith first, services later** | Speed to market early, autonomy later | Migration is long and risky |
| **Strong consistency on bookings** | A double booking ruins trust | Contention on very popular listings and dates |
| **ML-ranked search** | Higher booking rates | Needs data pipelines and constant experimentation |
| **Own data platform** | Reliable analytics and feature pipelines | Large team and infrastructure cost |

**WHAT IF THINGS BREAK?**

- The search index lags behind the calendar: guests may see a listing that was just booked, so the booking step re-checks availability transactionally.
- A payment provider is slow: bookings stay in a pending state and are completed or released by retries and webhooks.

**Toolbox concepts used here:** search index, geospatial index, retrieval and ranking, ACID transactions, idempotency, state machine, sharding, A/B testing

**CHECK YOURSELF**

1. Why is availability checked again at booking time?
2. What two stages does search use and why?
3. How does a hold with an expiry help during payment?
4. Why is a monolith a fine starting point?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design 'search as you drag the map' so results update within 300 ms. What do you cache and what do you recompute?

ARCHITECTURE TEARDOWN

# Discord

08

*Millions of people talking at once, each community served by its own tireless process.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Real-time chat and voice for communities | Huge servers with millions online at once | Elixir gateways, Cassandra to ScyllaDB, Rust | One lightweight process per community |

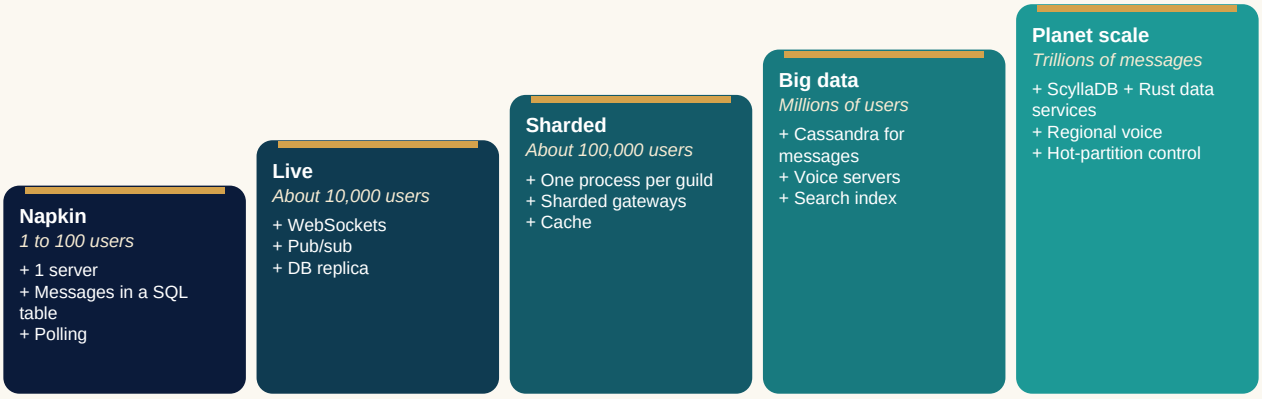
**THINK OF IT LIKE THIS**

A giant apartment building where each community has its own tireless doorman (a lightweight process). The doorman knows everyone inside, forwards every announcement to residents currently home, and the mailroom keeps all letters in order.

## The story in plain English

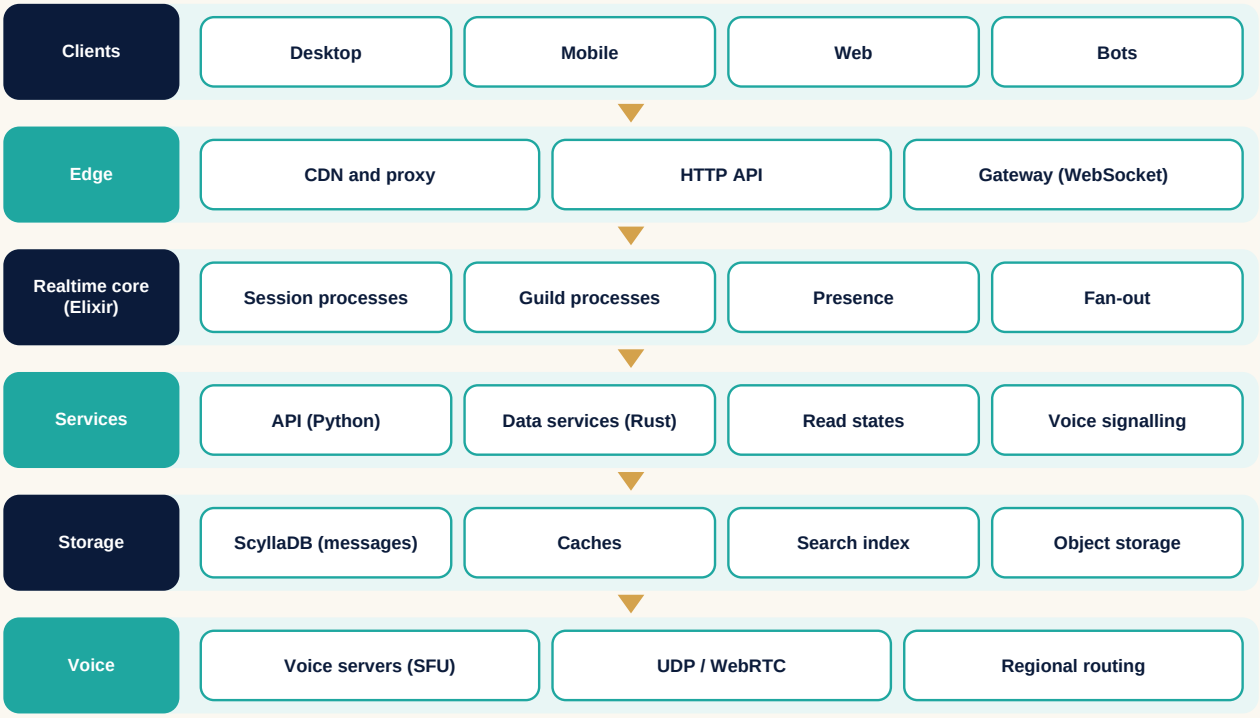
Clients keep a live **WebSocket** connection to a gateway. Each community (a 'guild') is managed by one lightweight Elixir/Erlang process that keeps its state and fans messages out to connected members. Messages are saved in a database built for huge write volumes, and voice runs separately over UDP through regional voice servers. Discord stores trillions of messages.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Polling is wasteful and slow | Persistent WebSocket connections | WebSocket gateway |
| **10K to 100K** | One process cannot track every community | One lightweight process per guild; shard gateways | Actor model (BEAM processes), sharding |
| **100K to millions** | Message table outgrows one database | A distributed wide-column store keyed by channel and time bucket | Cassandra, partitioning by (channel, bucket) |
| **Millions to planet** | Hot channels overload single partitions; JVM pauses spike latency | ScyllaDB, plus Rust services that coalesce duplicate reads | Request coalescing, hot partition protection |

## The big picture (high-end design)




*Simplified from Discord's engineering blog posts about Elixir, message storage and voice. Details evolve; the process-per-community and bucketed-storage ideas are the key lessons.*

## Deep dives

### A process per community (Elixir on the BEAM)

> **ANALOGY**
> Each community has its own doorman who never sleeps and never blocks the others.

**In plain English.** Elixir runs on the Erlang virtual machine, where millions of tiny isolated processes are cheap. Each guild is one process holding its state; each user session is another. A crash in one is contained and restarted by a supervisor.

#### Under the hood

- Actor model: processes talk by messages, no shared memory, so no locks.
- Fan-out: the guild process forwards an event to the sessions of online members.
- Large guilds use relays so one process is not responsible for every send.

### Messages at trillion scale: buckets and coalescing

> **ANALOGY**
> A mailroom with one pigeonhole per channel per week, so no single shelf gets absurdly deep.

**In plain English.** Messages are partitioned by channel and a time bucket. Very busy channels still created hot partitions, and Java garbage-collection pauses hurt latency, so storage moved to ScyllaDB with Rust services in front that merge identical simultaneous reads into one.

#### Under the hood

- Partition key (channel_id, bucket) keeps partitions bounded and time-ordered.
- Request coalescing: 1,000 users loading the same message get one database read.
- Consistent hashing routes related requests to the same data service.

## Voice: forward streams, do not mix them

**ANALOGY**
A switchboard that forwards each speaker's voice as it is instead of re-recording everyone into one tape.

**In plain English.** Voice travels over UDP using WebRTC. Servers act as selective forwarding units (SFUs): they relay each speaker's packets to the listeners rather than decoding and mixing them, which is much cheaper.

**Under the hood**

- UDP: a late audio packet is worthless, a lost one is barely heard.
- Regional voice servers keep distance, and so delay, small.
- The Opus codec adapts bitrate to the network.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Doorman for each community* | **Actor / lightweight process per guild** |
| *Live phone line to the building* | **WebSocket gateway connection** |
| *Pigeonhole per channel per week* | **Partition key with time bucket** |
| *Many people asking the same question, one answer* | **Request coalescing** |
| *Forward, do not re-record* | **SFU (selective forwarding unit)** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Elixir / BEAM for realtime** | Millions of cheap, isolated processes | A smaller talent pool and an unusual runtime |
| **Cassandra to ScyllaDB** | More predictable latency without JVM pauses | Migration effort; compaction and repair still need care |
| **Rust data services** | Efficiency and safe concurrency for hot paths | Another language to operate |
| **SFU voice** | Cheap relaying and low delay | Receivers must handle several streams |

**WHAT IF THINGS BREAK?**
- A guild process crashes: its supervisor restarts it and sessions resync state; other guilds are untouched.
- A viral channel gets hammered: coalescing and caches protect the database partition.

**Toolbox concepts used here:** WebSocket, actor model, fan-out, consistent hashing, partitioning, request coalescing, UDP real-time media, caching

**CHECK YOURSELF**

1. Why is one lightweight process per community a good fit?
2. What problem do time buckets solve in message storage?
3. How does request coalescing protect a hot partition?
4. Why do voice calls prefer UDP?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design 'user is typing...' indicators for a server with 500,000 members online. How do you avoid a message storm?

ARCHITECTURE TEARDOWN

# Stripe

09

*Moving money correctly, exactly once, every time.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| **Payments infrastructure and APIs** | **Never charging twice or losing a cent** | **Idempotency keys, API versioning, rate limiters** | **Correctness first, then speed** |

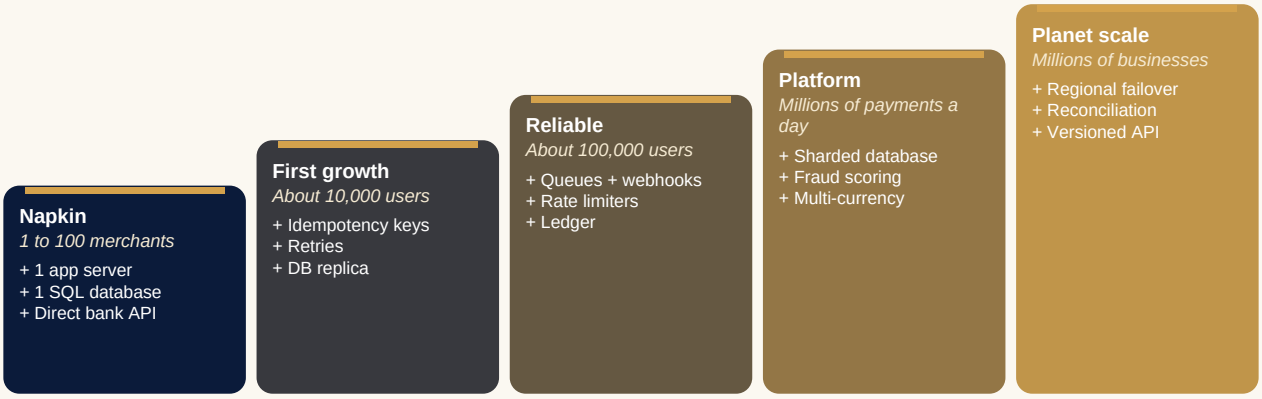
**THINK OF IT LIKE THIS**

A bank teller who stamps every transaction with a unique receipt number, so doing it twice is impossible, and who writes everything in a permanent ledger book that can always be checked against the bank's own records.

## The story in plain English

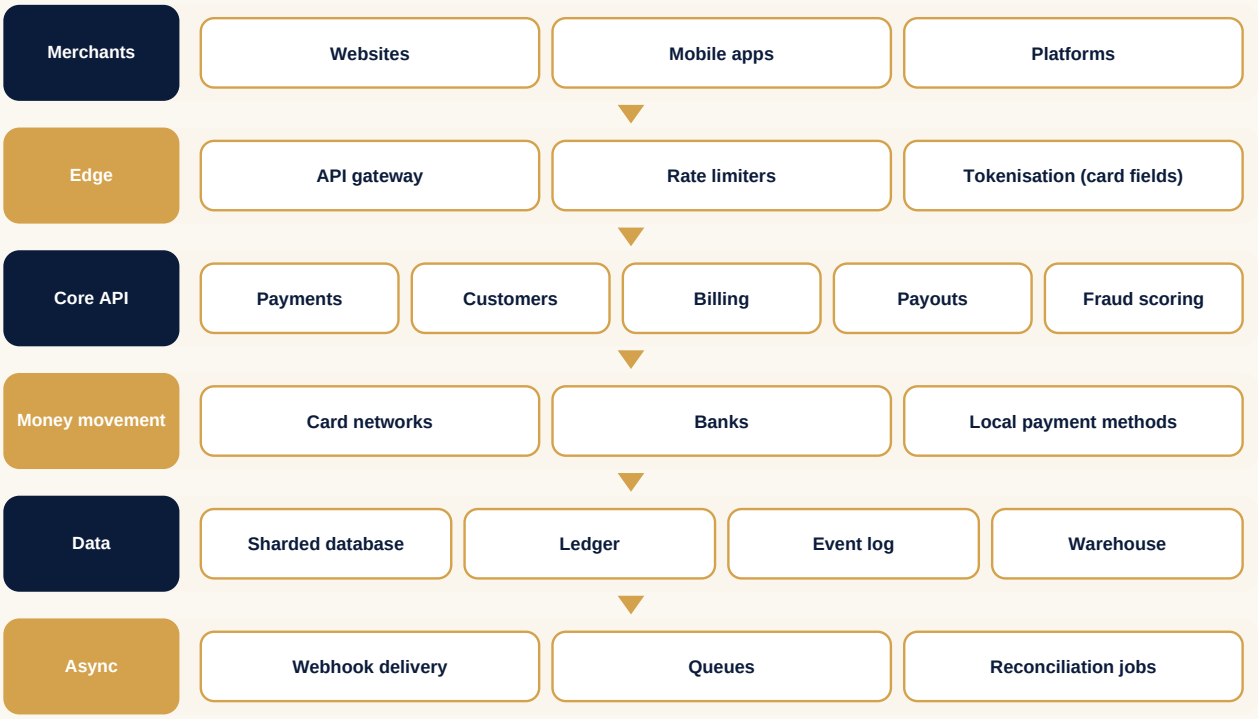
A card payment travels a chain: the customer's browser, the merchant, Stripe, the card network and the customer's bank; money settles days later. Each hop can fail or time out, yet the result must be exactly one charge. Stripe's design therefore centres on **idempotency**, durable state machines, a double-entry **ledger** and careful **reconciliation**.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **1K to 10K** | Network timeouts cause retries and double charges | Idempotency keys on every write | Idempotency |
| **10K to 100K** | One noisy client can slow everyone down | Rate limiters and queues; async webhooks | Rate limiting, load shedding, webhooks |
| **100K to millions** | Balances must always add up | Double-entry ledger and a sharded, strongly consistent store | Ledger, ACID, sharding |
| **Millions to planet** | Old integrations must keep working for years | Date-based API versions pinned per account; nightly reconciliation | API versioning, reconciliation jobs |

## The big picture (high-end design)




*Simplified from Stripe's public engineering blog posts (idempotency, rate limiters, ledger, API versioning). Internal specifics are not fully public.*

## Deep dives

### Idempotency keys: safe to retry

> **ANALOGY**
> A cinema ticket with a unique serial number: showing it twice at the door admits you only once.

**In plain English.** A client sends a unique key with every payment request. Stripe stores the key and the result. If the same key arrives again (a retry after a timeout), it returns the stored result instead of charging again.

#### Under the hood

- Keys are saved with the request fingerprint; a reused key with different data is rejected.
- Results are stored for a window (for example 24 hours).
- Clients retry freely with backoff because retries are now safe.

### Rate limiters and load shedding

> **ANALOGY**
> Bouncers at the door with different jobs: some limit entry per minute, some cap how many are inside, some turn away guests when the club is nearly full.

**In plain English.** Stripe described several layers: a per-user request rate limiter, a concurrency limiter, a fleet-wide load shedder that drops low-priority traffic, and a worker-utilisation shedder for critical traffic.

#### Under the hood

- Token bucket for request rates, backed by Redis counters.
- Critical requests (charges) are protected over less critical ones (listing old data).
- Limits return HTTP 429 and clients back off.

## The ledger and reconciliation

> **ANALOGY**
> A bookkeeper recording every coin movement in two columns, then checking the books against the bank statement each night.

**In plain English.** Every movement of money creates balanced debit and credit entries in an append-only ledger. Balances are derived from entries. Reconciliation jobs compare Stripe's records with partners' files to find mismatches.

**Under the hood**

- Append-only records are auditable and never silently edited.
- State machines (created, authorised, captured, refunded) with idempotent transitions.
- Mismatches raise alerts instead of being guessed away.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Unique receipt number* | **Idempotency key** |
| *Bouncers* | **Rate limiters and load shedders** |
| *Two-column bookkeeping* | **Double-entry ledger** |
| *Check books against the bank statement* | **Reconciliation** |
| *Pin each customer to a menu edition* | **Date-based API versioning per account** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Idempotent write API** | Retries are inevitable on networks | Extra storage and key handling logic |
| **Strong consistency for money** | Wrong balances are unacceptable | Lower throughput and costlier infrastructure |
| **Versioned API per account** | Integrations never break unexpectedly | Old versions must be maintained and translated |
| **Priority-based load shedding** | Protect critical payments in overload | Some traffic is knowingly rejected |

> **WHAT IF THINGS BREAK?**
> - A bank partner times out after charging: the payment stays 'pending', and webhooks or reconciliation settle the true status.
> - A merchant's server is down: webhook deliveries retry with backoff for days and never block others.

**Toolbox concepts used here:** idempotency, ACID, ledger, state machine, rate limiting, load shedding, webhooks, retries with backoff, reconciliation

**CHECK YOURSELF**

1. How does an idempotency key stop a double charge?
2. Why use several kinds of limiters instead of one?
3. Why is the ledger append-only?
4. What does reconciliation catch that code cannot?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design webhook delivery with retries for 100,000 merchants so one slow merchant never delays the others.

ARCHITECTURE TEARDOWN

# Zoom

10

*Face-to-face video for millions at once, where late is worse than lost.*

| WHAT IT IS | HARDEST PROBLEM | FAMOUS FOR | CORE IDEA |
|---|---|---|---|
| Video meetings and webinars | Low-delay video for many people on any network | Multimedia routers; 10M to 300M in months | Forward streams, do not mix them |

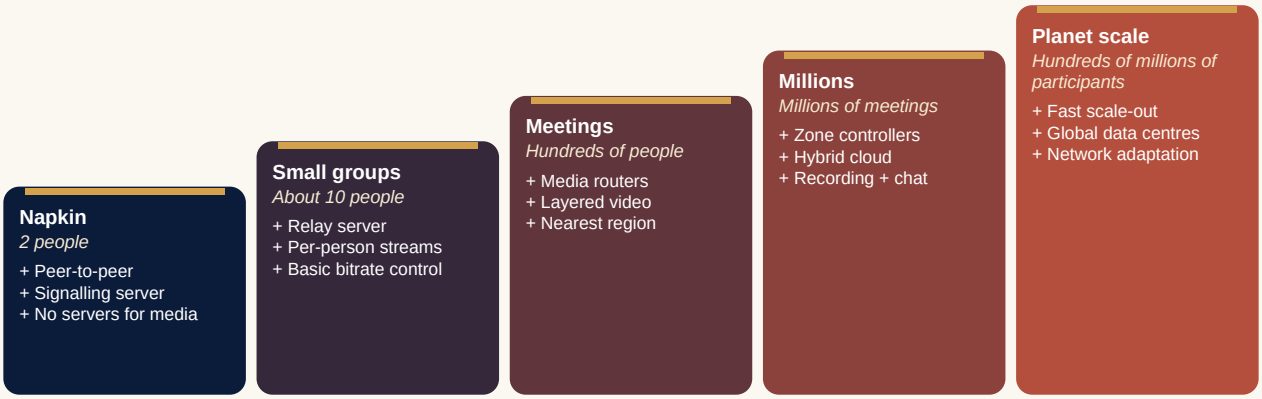
**THINK OF IT LIKE THIS**

A switchboard operator forwarding phone calls rather than someone re-recording everyone's voices onto one tape. Each speaker's stream is passed on as it is, and each listener's device decides what to show.

## The story in plain English

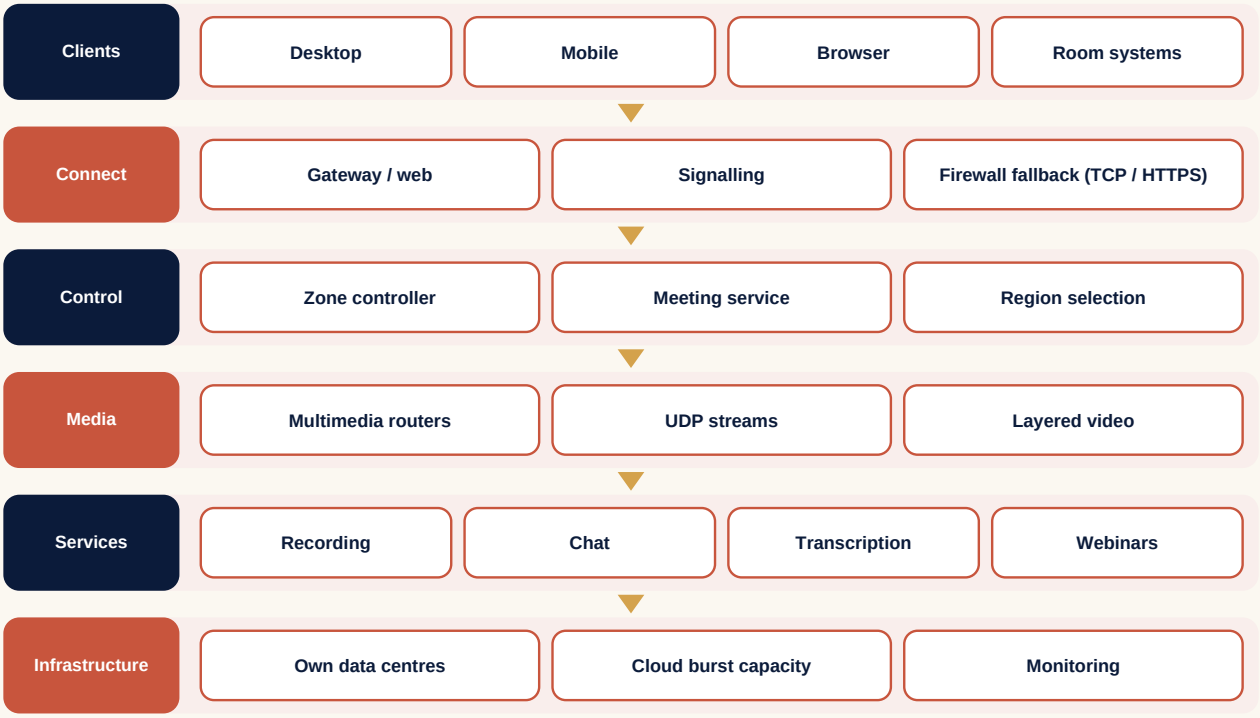
In a video call every person sends one stream. Mixing everyone into one picture on a server is very expensive. Zoom-style systems use media routers that **forward** streams to participants, choosing a quality level per receiver. Streams travel over **UDP** for low delay, and the system connects each person to a nearby data centre. In early 2020 daily meeting participants grew from about 10 million to over 300 million within a few months.

## The scale ladder: from one server to planet scale




| Growing from | What breaks | What gets added | Engineer word |
|---|---|---|---|
| **2 to 10 people** | Direct connections multiply with every person and fail behind firewalls | A server relays the streams | Relay / TURN server, SFU idea |
| **10 to 100 people** | Everyone sending to everyone overloads upload bandwidth | Each person sends one stream to a router that forwards it | Selective forwarding |
| **100 to millions** | Different devices and networks need different quality | Layered video; router drops layers per receiver | Scalable video coding (SVC), adaptive bitrate |
| **Millions to planet** | Demand jumped 30 times in months | Hybrid cloud, quick scale-out, nearest-region routing | Autoscaling, hybrid cloud, geo routing |

## The big picture (high-end design)




*Simplified from Zoom's public architecture talks and blog posts. Specifics change; the forward-not-mix design is the stable idea.*

## Deep dives

### UDP: why late is worse than lost

> **ANALOGY**
> A live radio show: a missing word is fine, but a delayed word makes the whole conversation awkward.

**In plain English.** Real-time media uses UDP, which sends packets with no waiting and no retransmission. Zoom prefers UDP and falls back to TCP or HTTPS when firewalls block it.

#### Under the hood

- No head-of-line blocking: one lost packet does not stall later audio.
- The application adds its own gentle protection: forward error correction and selective retransmission.
- Jitter buffers smooth out uneven arrival times.

### Forwarding layered video (SVC)

> **ANALOGY**
> A recipe card with the base dish on top and optional extras below. Each guest gets as many layers as their plate can hold.

**In plain English.** A sender encodes video in layers: a basic layer plus enhancement layers. The router forwards all layers to strong receivers and only the base layer to weak ones, without re-encoding.

#### Under the hood

- One upload serves many different receivers.
- The router reacts to each receiver's bandwidth in real time.
- Large meetings show only the active speaker in high quality.

## Joining a meeting and scaling 30 times

**ANALOGY**
A festival that adds gates and staff in hours, and sends each ticket holder to the nearest entrance.

**In plain English.** A controller assigns each participant to a nearby data centre and a router with spare capacity. In 2020 Zoom expanded its own data centres and burst into public clouds to keep up.

### Under the hood

- Geo and load-aware routing keeps delay low.
- Stateless control services and autoscaling make quick growth possible.
- Capacity monitoring and gradual traffic shifting avoid overload.

### Say it plain, then say it like an engineer

| In everyday words | Engineers call it |
|---|---|
| *Forward, do not re-record* | **Selective forwarding (SFU-style routing)** |
| *Late is worse than lost* | **UDP for real-time media** |
| *Base dish plus extras* | **Scalable video coding (layers)** |
| *Smooth uneven arrival* | **Jitter buffer** |
| *Nearest entrance* | **Geo and load-aware routing** |

### Key decisions and the price they pay

| Decision | Why they did it | The price they pay |
|---|---|---|
| **Forward, do not mix** | Mixing is expensive and adds delay | Receivers must decode several streams |
| **UDP with TCP fallback** | Lowest delay where allowed | Fallback paths are slower and costlier |
| **Hybrid cloud** | Fast capacity during spikes | Complexity and cost across providers |
| **End-to-end encryption as an option** | Strong privacy for those who need it | Disables some cloud features (recording, transcription) |

**WHAT IF THINGS BREAK?**
- A router fails mid-meeting: clients reconnect to another router and resume within seconds.
- A global event overloads one region: new participants are routed to other regions while existing meetings stay put.

**Toolbox concepts used here:** UDP and real-time media, selective forwarding, adaptive bitrate, autoscaling, geo routing, hybrid cloud, graceful fallback

**CHECK YOURSELF**

1. Why not just mix everyone's video on the server?
2. Why does Zoom prefer UDP, and what is the fallback?
3. How does layered video let weak and strong devices share one stream?
4. How did Zoom handle a 30x jump in demand?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

**REDESIGN CHALLENGE**

Design a webinar for 50,000 viewers and 5 presenters. Which parts need real-time delivery and which can use a CDN?

# ALL
## Across All Ten

*Patterns, comparisons and your boss-level exam.*

- Which concept shows up where
- Side-by-side comparison
- Same problem, different answers
- Ten lessons
- Boss-level exam
- Design your own

PATTERN MAP

# Which concept shows up where

A bigger dot means the idea is central to that company's design. Notice how few ideas explain so many giants.

| CONCEPT | NFX | UBR | TGM | AMZ | GGL | SPT | ABB | DSC | STR | ZOM |
|---|---|---|---|---|---|---|---|---|---|---|
| CDN and edge delivery | ● | · | ● | ● | · | ● | ● | ● | · | · |
| Microservices | ● | ● | · | ● | · | ● | ● | ● | ● | · |
| Queues and event streaming | ● | ● | · | ● | · | ● | ● | ● | ● | · |
| Consistent hashing | ● | ● | · | ● | · | ● | · | ● | · | · |
| Geospatial index | · | ● | · | · | · | · | ● | · | · | · |
| Inverted index and search | · | · | ● | ● | ● | ● | ● | ● | · | · |
| WebSockets and live connections | · | ● | ● | · | · | ● | · | ● | · | ● |
| UDP real-time media | · | · | · | · | · | · | · | ● | · | ● |
| Idempotency | · | ● | · | ● | · | · | ● | · | ● | · |
| Saga and distributed transactions | · | ● | · | ● | · | · | ● | · | ● | · |
| Rate limiting and load shedding | · | ● | · | ● | · | · | · | ● | ● | · |
| Circuit breaker and fallbacks | ● | ● | · | ● | · | ● | · | ● | · | · |
| ML ranking and recommendations | ● | ● | · | ● | ● | ● | ● | · | · | · |
| Adaptive quality (bitrate / layers) | ● | · | · | · | · | ● | · | · | · | ● |
| Sharded or relational ACID | · | ● | · | ● | · | · | ● | · | ● | · |
| Eventual consistency stores | ● | · | · | ● | · | ● | · | ● | · | · |
| Multi-region / multi-DC | ● | ● | ● | ● | ● | ● | · | · | · | ● |
| Scatter-gather and tail latency | · | · | · | · | ● | · | ● | · | · | · |
| Delta sync and event offsets | · | · | ● | · | · | · | · | ● | · | · |
| Chaos and failure drills | ● | · | · | ● | · | · | · | · | · | · |

**Key:** NFX = Netflix | UBR = Uber | TGM = Telegram | AMZ = Amazon | GGL = Google Search | SPT = Spotify | ABB = Airbnb | DSC = Discord | STR = Stripe | ZOM = Zoom

COMPARE

# Side-by-side comparison

| Company | Core challenge | Signature trick | Consistency stance |
|---|---|---|---|
| **Netflix** | Huge video traffic | Own CDN inside ISPs; control vs data plane | Eventual for most data; strong where needed |
| **Uber** | Real-time matching | H3 cells, in-memory index, consistent-hash ring | Strong for trip state; eventual for locations |
| **Telegram** | Multi-device sync, giant channels | Update counters, pull-based fan-out | Server-ordered per account |
| **Amazon** | Never lose an order | Service per team; Dynamo-style cart; saga checkout | Available cart, strong payment and stock |
| **Google Search** | Search the web in ms | Sharded inverted index, hedged requests | Eventual freshness |
| **Spotify** | Instant playback, discovery | Chunking, prefetch, offline recommendations | Eventual; cloud state for Connect |
| **Airbnb** | Unique listings and dates | Retrieve then rank; transactional calendar | Strong for bookings |
| **Discord** | Huge live communities | Process per guild; bucketed storage; coalescing | Per-channel ordering; eventual elsewhere |
| **Stripe** | Exactly-once money movement | Idempotency keys, ledger, reconciliation | Strong for money |
| **Zoom** | Low-delay video for many | Forward not mix; UDP; layered video | Real-time, best effort |

## Same problem, different answers

The cleverest way to learn is to compare how different companies solve the *same* sub-problem.

| Sub-problem | How different giants solve it |
|---|---|
| **Real-time delivery to clients** | Uber: push over live connections. Telegram: persistent connection plus counters for missed updates. Discord: WebSocket gateway and a process per guild. Zoom: UDP media routers. |
| **Huge audiences** | Telegram: store once, pull on read. Netflix: pre-place content at the edge. Spotify: cache chunks at a CDN. Discord: fan-out through guild relays. |
| **Finding 'near me' or 'best match'** | Uber: H3 cells then ETA ranking. Airbnb: geo retrieval then ML ranking. Google: sharded inverted index then re-ranking. |
| **Never doing it twice** | Stripe: idempotency keys. Amazon: idempotent saga steps. Airbnb: unique night constraint. Telegram: counters make sync idempotent. |
| **Surviving failure** | Netflix: chaos drills and regional evacuation. Amazon: cells and static stability. Discord: supervised processes. Google: hedged requests. |
| **Storing massive data** | Netflix and Amazon: Dynamo-style / Cassandra-style stores. Discord: bucketed partitions on ScyllaDB. Stripe: sharded strongly consistent store. Spotify: object storage plus wide-column. |

TAKEAWAYS

# Ten lessons from ten giants

| # | Lesson | Why it matters |
|---|---|---|
| 1 | Start simple, climb the ladder | Every giant began with one server. Add a component only when a measured problem demands it. |
| 2 | Separate the brain from the muscle (Netflix) | Keep decisions in one place and heavy data movement at the edge. |
| 3 | Keep hot, short-lived state in memory (Uber) | Locations, presence and sessions do not belong on slow disks. |
| 4 | Store once, pull many (Telegram) | For huge audiences, avoid copying data into millions of inboxes. |
| 5 | Match consistency to the cost of being wrong (Amazon) | Carts favour availability; payments and stock favour correctness. |
| 6 | Split, ask everyone, merge, manage the tail (Google) | Scatter-gather needs tricks for the slowest machine. |
| 7 | Precompute what you can (Spotify) | Heavy ML offline, quick lookups online; prefetch what users will want next. |
| 8 | Retrieve then rank, and never double-book (Airbnb) | Cheap candidate search first, expensive judging second, transactions for scarce resources. |
| 9 | Contain failures (Discord, Amazon) | Small isolated processes and cells stop one problem becoming everyone's problem. |
| 10 | Make writes safe to repeat (Stripe) | Idempotency turns scary retries into a feature. |

**REAL WORLD**

Zoom's real-time rule rounds this off: for live media, **late is worse than lost**. Always ask what a late or wrong answer costs in your product.

BOSS LEVEL

# If you can answer these without hesitation, you are ready for the next round

Twenty questions across all ten architectures. Answer out loud, in simple words first, then add the technical terms. Score yourself honestly.

### BOSS LEVEL: PART 1

1. Netflix: why split the control plane from the data plane?
2. Netflix: how does Open Connect lower cost and latency?
3. Uber: why in-memory geo indexing, and why hexagons?
4. Uber: how do you stop two riders being matched to one driver?
5. Telegram: how do update counters let a device sync after being offline?
6. Telegram: why is pull better than push for a million-subscriber channel?
7. Amazon: why is the cart AP while payment is strongly consistent?
8. Amazon: how does a saga with idempotent steps handle a failure at shipping?
9. Google: why is query latency decided by the slowest shard, and what is a hedged request?
10. Google: why rank in two stages?

*Answer out loud, without peeking. Hesitation shows you which chapter to re-read.*

### BOSS LEVEL: PART 2

1. Spotify: how do chunking, prefetch and caching make 'next' feel instant?
2. Spotify: why compute Discover Weekly offline?
3. Airbnb: how do you guarantee a night is never sold twice?
4. Airbnb: what do retrieval and ranking each do?
5. Discord: why is a process per community a good fit, and what do time buckets fix?
6. Discord: what is request coalescing?
7. Stripe: how does an idempotency key stop a double charge?
8. Stripe: why an append-only ledger and nightly reconciliation?
9. Zoom: why forward streams rather than mix them?
10. Zoom: why UDP, and what happens when a firewall blocks it?

*Score: 18 to 20 = interview ready. 14 to 17 = review the chapters you stumbled on. Under 14 = reread the toolbox, then retry.*

DESIGN YOUR OWN

# Borrow from the giants

Pick a product below, name the teardowns you would borrow from, and design it on a blank page in 45 minutes.

| Prompt | Borrow from | Key question to answer |
|---|---|---|
| **Twitch (live streaming)** | Zoom, Netflix, Discord | Low-delay ingest and CDN delivery, plus live chat at huge scale |
| **Google Maps** | Uber, Google Search | Road graph routing, live traffic and tile delivery |
| **DoorDash** | Uber, Airbnb, Stripe | Three-sided matching, ETA promises and payments |
| **Slack** | Discord, Telegram | Workspaces as shards, message history, search and presence |
| **TikTok** | Netflix, Spotify | Short video delivery and a recommendation feed with fast feedback |
| **Booking.com** | Airbnb, Amazon | Availability, overbooking policy and search ranking |

WHAT NEXT?

# Now build your own.

*Real understanding comes from drawing, estimating and explaining, again and again.*

- Redraw each company's scale ladder from memory.
- Explain every analogy to a friend, then add the engineer words.
- Pick a Redesign Challenge and design it on a blank page in 45 minutes.
- Combine two companies' tricks for a new product (for example Twitch = Zoom + Netflix).
- Revisit Module 25 for the fundamentals and practice problems.


## Sheharyar Sarmad

LinkedIn: linkedin.com/in/sheharyar-sarmad-9b7736289

GitHub: github.com/Sheharyar-Sarmad/ai-zero-to-hero


Module 25: 25_systemdesign_foundation

*If this helped you, star the repo, share it with someone preparing for interviews, and tell me which architecture to break next.*